TUTORIAL FOR COMPUTER PROGRAMMING

2025/2026

RODNEY PETRUS BALANDONG

Contents

Preface

Apart from the lecture materials, this tutorial series is designed to provide a practical, hands-on introduction to computer programming using Python. The focus is on building a strong foundation in programming concepts while applying them to a consistent dataset—the Iris dataset. This approach allows students to see the progression of their skills as they work through increasingly complex tasks, ultimately leading to the development of a rule-based classifier.

Goal and Strategies

The primary goal of this tutorial series is to transition students from basic syntax to building functional, rule-based classifiers. The strategy employed is one of incremental complexity:

- **Continuity:** We use the same dataset (Iris) across all sessions to reduce cognitive load and allow focus on new programming concepts.
- **Scaffolding:** Each session builds directly on the previous one, starting with simple variables and progressing to functions, loops, and modular design.
- **Practical Application:** Theoretical concepts are immediately applied to solve concrete problems, reinforcing learning through doing.
- To avoid installation issues, we will use Github Codespaces for all coding exercises, ensuring that students can focus on learning rather than troubleshooting their development environment.

Intended Audience

This material is written for:

- Undergraduate students in computer science, data science, or related engineering fields.
- Beginners with little to no prior programming experience who want a structured introduction to Python.
- Learners interested in the intersection of programming and introductory data analysis.

Acknowledgements

This tutorial series is the first edition,and it is a work in progress. Therefore, I welcome feedback and suggestions for improvement. If you have any comments or ideas on how to enhance the content, please feel free to reach out to me through the project's GitHub repository.

Changelog

Mock Example

Version 0.1

- Initial draft of the tutorial book.
- Added preface and early session material.

Tutorial Material

GitHub Account Registration

Before starting this course, every student must create a GitHub account. GitHub will be used throughout the semester to access course materials, fork repositories, and run code using GitHub Codespaces.

Create a GitHub account

Open the following registration link in your browser click_me: GitHub Sign Up. You should see a page similar to the one shown in Figure 1.

Then complete the registration form carefully by following these steps:

1. In the **Email** field, use your official UMS email account.

   For example:

   ```
   ggg@iluv.ums.edu.my
   ```

   where `ggg` should be replaced with your own UMS username.

2. In the **Password** field, create a strong password.

   Your password should:

   - contain a combination of uppercase and lowercase letters,
   - include numbers and symbols,
   - not be too short,
   - not be something easy to guess such as your name or matric number.

3. In the **Username** field, choose a good and professional username.

   Please choose carefully because this account may be used in the future for public collaboration, portfolio work, open-source contribution, or job-related projects. A good username should:

   - be simple,
   - be professional,
   - be easy to remember,
   - avoid unnecessary symbols or random numbers unless needed.

   Examples of suitable usernames:

   - `ahmadrahman`
   - `nuraisyah`
   - `danielums`
   - `farhan.dev`

4. Set the **Country** to **Malaysia**.

5. Continue with the registration process by following all instructions shown in the browser.

6. GitHub may ask you to complete one or more verification steps, for example:

   - solving a simple puzzle,

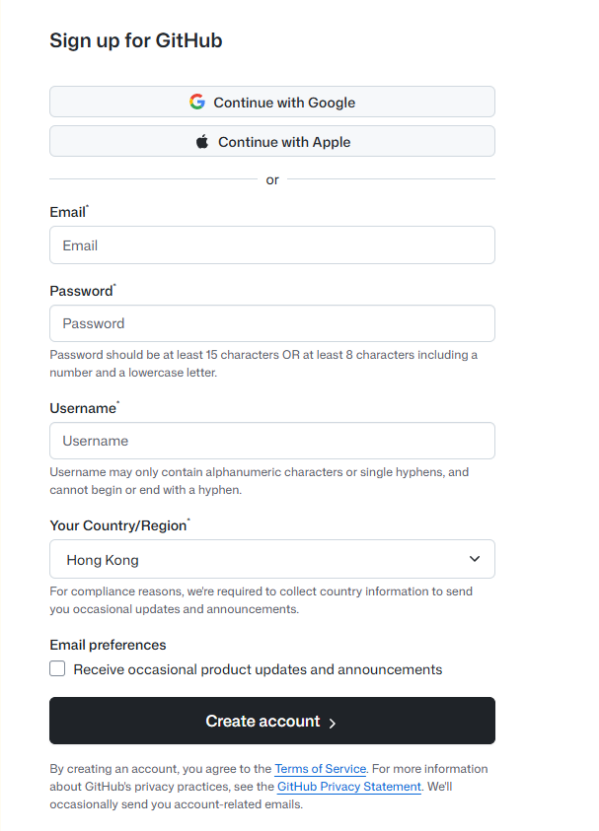

Figure 1: GitHub Sign Up Page

- clicking a verification checkbox,
- confirming that you are not a robot,
- verifying your email address.

7. Read each instruction carefully and complete the verification until GitHub allows you to proceed.
8. If you do not know what to do during any verification step, please ask my FYP student for assistance (YASMIN MADISIN REKY,RACHEL LINDA JANILON,MOHAMAD SUHAIMIE,FARRAH NABILA BINTI MAIDIN).
9. Once all required information has been filled in and verification has been completed, finish the sign-up process until your account is created successfully.

Verify that the account has been created successfully

After registration, you must make sure that your GitHub account is really working.

1. Log out first if GitHub logs you in automatically after sign-up.
2. Go to the GitHub login page and sign in using the account that you have just created.
3. Enter your registered email address and password.
4. If login is successful, you should be able to enter your GitHub account homepage.
5. You should also be able to see your profile icon or account menu on the top part of the page.
6. This means that your GitHub account has been created successfully and is ready to use.

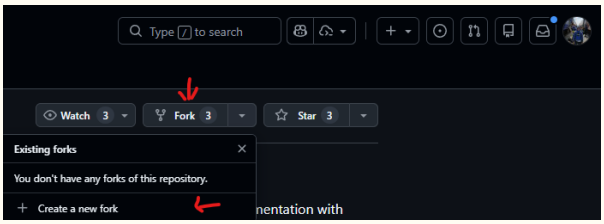

Figure 2: Fork Button Location

7. If login fails, check the following:
   - whether the email address is correct,
   - whether the password is correct,
   - whether email verification is still pending.

Course Repository

Open the following repository link click_me: Course Repository.

This repository is used as a companion resource for the **Basic Programming** tutorial sessions. It contains code examples, exercises, and supplementary materials to support students throughout the semester. The content will be updated from time to time.

What is a fork?

For students who are using GitHub for the first time, it is important to understand what a **fork** is.

A **fork** is your own copy of someone else's repository under your personal GitHub account. When you fork a repository:

- you get a copy of the original repository,
- the copy belongs to your GitHub account,
- you can work on it without changing the original repository directly,
- you can later sync your fork when the original repository is updated.

In this course, each student needs to fork the repository so that everyone has their own copy to work with.

How to fork the repository

1. Open the repository page click_me: Course Repository.
2. Look at the top area of the repository page and find the **Fork** button as shown in Figure 2.
3. Click the **Fork** button.Kindly refer to Figure 2 for the location of the Fork button.
4. A small menu or page should appear. For first-time users, GitHub may show a message similar to:

   ```
   Existing forks
   You don't have any forks of this repository.
   Create a new fork
   ```

5. Click **Create a new fork**.

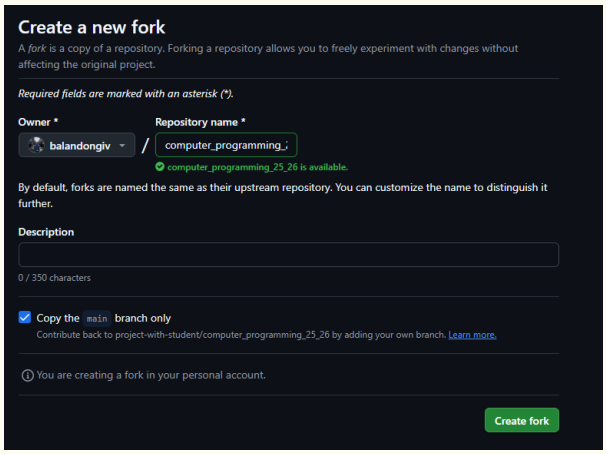

Figure 3: Create Fork Page

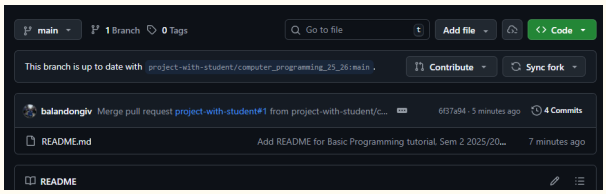

Figure 4: Forked Repository Page

6. GitHub will then open the fork creation page as shown in 3.
7. You may rename the repository to whatever you like. However, you may also keep the repository name as it is, which is usually the easiest option.
8. After checking the repository name, click the **Create fork** button.
9. Wait for GitHub to create the fork.
10. A new page should appear after the forking process is completed.
11. If you notice carefully, the page layout should look almost the same as the original repository page. This means that you now have an exact copy of the original repository under your own GitHub account as shown in Figure 4.
12. You should also be able to see that the repository now belongs to your account, not only to the original owner.

Why the fork is important

Forking is important in this course because:

- you will have your own working copy of the repository,
- you can use Codespaces from your own copy,
- you can save your own progress safely,
- you can receive future updates from the original repository by syncing your fork.

Codespaces

For this course, at the beginning, we will be using GitHub Codespaces.

GitHub Codespaces is a cloud-based development environment provided by GitHub. It allows students to write and run code directly in the browser without needing to perform full software installation on their own computers.

Why we use GitHub Codespaces

The main advantage is that we do not need to do complicated setup at the beginning. Apart from that, GitHub Codespaces also provides other benefits:

- **No local installation is needed.** Students do not need to install many tools manually before starting.
- **Same environment for everyone.** This reduces problems caused by different operating systems or software versions.
- **Accessible from anywhere.** Students can open their work from different computers as long as they can log in to GitHub.
- **Faster to start.** Students can focus on learning programming instead of spending too much time on configuration.
- **Better consistency.** The required packages and dependencies can already be prepared in advance.
- **Convenient for class use.** This makes it easier to follow tutorial sessions because everyone works in a similar environment.

How to open Codespaces

After you have successfully forked the repository, open **your forked repository**, not the original repository.

Then follow these steps:

1. In your forked repository page, click the **Code** button. The button and UI is as shown in Figure 5.
2. A menu will appear with several tabs.
3. Click the **Codespaces** tab.
4. Then click **Create codespace on main**.
5. After that, GitHub will start preparing the environment as shown in Figure 6.
6. You will see a message or page indicating that Codespaces is opening.
7. Wait until the Codespace finishes loading completely .
8. Once it has opened, an automatic setup process may run for a while, about 10 minutes or more, depending on the server load and your internet connection.
9. After the setup is complete, you should see the Codespaces interface as shown in Figure 7.

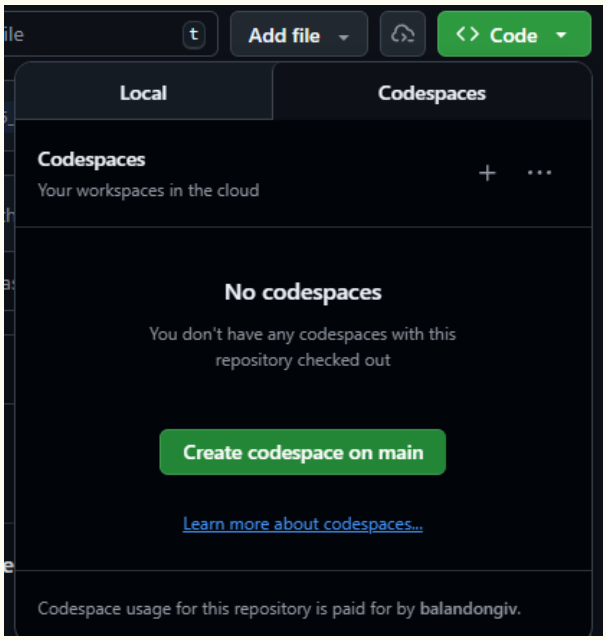

Figure 5: Codespaces Tab Location

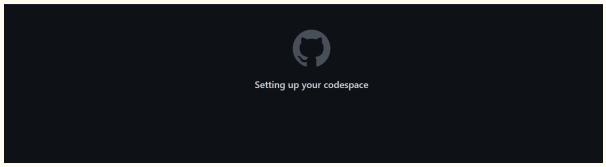

Figure 6: Loading Codespaces

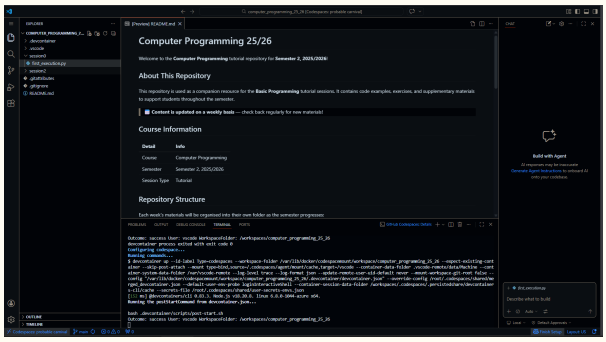

Figure 7: Codespaces Successfully Opened

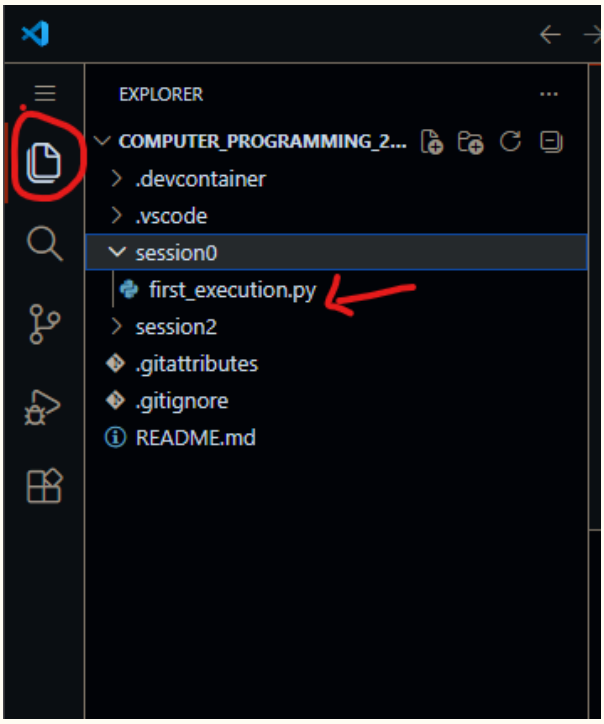

Figure 8: Running the Initial Check Script

Initial check inside Codespaces

All the necessary packages should already have been installed for you.

To make sure everything is working correctly, first click the "Explorer" icon on the left sidebar to open the file explorer. Then, locate the folder name "session0" and open it. Inside that folder, you should find a file named "first_execution.py". Double click it. Then, click the "Run" button at the top right corner of the editor as shown in Figure 8.

The output should appear in the 9

If something does not work

If the Codespace does not open properly, or if the script cannot be run:

- check that you opened Codespaces from your own fork,

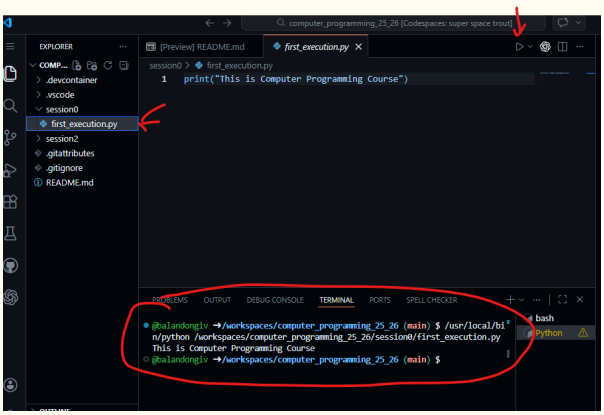

Figure 9: Running the Initial Check Script

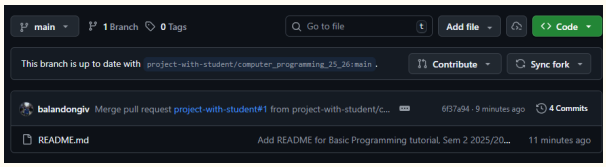

Figure 10: Sync Fork Button Location

- refresh the page and wait again,
- make sure the terminal is active,
- try running the command again carefully,
- ask for help if you are not sure what went wrong.

Pulling new updates from the original repository

As mentioned earlier, I will update the content of the repository from time to time. Therefore, you need to **sync** your fork so that your copy receives the latest updates.

How to sync the fork on GitHub

1. Open your forked repository on GitHub.
2. Near the top area of the repository page, find the **Sync fork** button.
3. Click **Sync fork**. The button and UI is as shown in Figure 10.
4. If updates are available from the original repository, GitHub will give you the option to update your fork.
5. Proceed with the sync action until your fork is updated.
6. After the sync is successful, your fork will contain the latest content from the original repository.

How to update your Codespace after syncing

If your Codespace is already open when you sync the fork on GitHub, you should also update the files inside the Codespace.

1. Return to your Codespace terminal.
2. Run:

   ```
   git pull
   ```

3. Wait until the latest changes are downloaded into your Codespace.
4. If needed, run `first_run.py` again to make sure everything is still working correctly.

Important reminder

Throughout the semester, always remember the following workflow:

1. Use your own GitHub account.
2. Work from your own fork of the repository.

3. Open Codespaces from your fork.
4. Sync your fork whenever new course content is released.
5. Pull the latest changes in Codespaces after syncing.

How to Stop a GitHub Codespace to Save CPU/Compute Time

To stop GitHub Codespaces from using compute or CPU time, you should **stop the codespace** when you are not using it. A stopped codespace keeps your saved work, but all running processes are halted, so compute usage stops.

Ways to Stop a Codespace

- **In the browser:** Open your codespace and choose **Stop codespace**.
- **In VS Code:** Open the Command Palette and select `Codespaces: Stop Codespace`.
- **Using the CLI:**

```
gh codespace stop
```

Then choose the codespace you want to stop.

Important Details

- **Idle timeout:** Codespaces automatically stop after a period of inactivity. The default timeout is usually 30 minutes, and you can change this in your Codespaces settings for new codespaces.
- **Storage still remains:** Stopping a codespace saves compute time, but the codespace still uses storage until it is deleted. If you no longer need it, delete it to avoid storage charges.

Recommended Workflow

1. Commit and push your changes.
2. Stop the codespace.
3. Delete old unused codespaces to avoid storage costs.

Session 1

Why Just Not Use AI for Coding

Section 1.1 Replicating a Simple Machine Learning Workflow

Imagine you are a software developer working under my supervision. I have recently come across an exciting demonstration of a machine learning model in action which is accessable via the link. Impressed by its functionality, I want you to replicate a simplified version of this machine learning workflow.

1.1.1 Scenario 1: Initial Attempt - Can LLMs Do It All?

As a first step, I want to see how effectively you can utilize readily available Large Language Models to achieve this task, even if you have limited to no prior coding experience.

Your Task: Explore LLM Capabilities with a Simple Prompt

1. **Choose an LLM:** Select any Large Language Model platform you are comfortable with, such as ChatGPT, Gemini, DeepSeek Coder, or any other similar service.
2. **Observe the Machine Learning Example:** Carefully review the provided example
3. **Formulate an Initial Prompt:** Based on your observation of the machine learning workflow, write a prompt for your chosen LLM. This prompt should be as natural and straightforward as possible, reflecting an initial attempt by someone with limited coding knowledge who believes in the "magic" of AI. For example, you might write something like:

   `Write code to train a machine learning model that classifies handwritten digits.`
4. **Generate Code:** Input your prompt into the LLM and request it to generate the code. Since I am comfortable in Python, you should restrict to provide the output code in Python. But, you may ask how to run the code tho and ask the LLM to also suggest any necessary libraries it deems appropriate for this task.
5. **Execute the Code:** Attempt to execute the code generated by the LLM. Note down any errors, issues, or the level of success you achieve.
6. **Analyze the Outcome:** Reflect on the results. Did the LLM produce functional code? How closely does it resemble a complete machine learning workflow? What are the shortcomings of this initial approach?

1.1.2 Scenario 2: Detailed Instructions - A Helping Hand

Let's assume your initial attempt in Scenario 1 did not produce the desired, detailed machine learning workflow. As your supervisor, recognizing the potential of LLMs but also the need for more specific guidance, I am now providing you with a much more detailed instruction set in the form of a comprehensive prompt.

Your Task: Refine Workflow with a Detailed Prompt

So to begin,

1. **Access Detailed Prompt:** Navigate to the following link to access a detailed prompt specifically designed for generating a machine learning workflow: Detailed Prompt Gist.
2. **Understand the Detailed Prompt:** Carefully read and understand the detailed prompt provided. Notice how it breaks down the workflow into components, specifies preprocessing and training steps, and describes evaluation and visualization elements.
3. **Use the Detailed Prompt with LLM:** Paste the entire detailed prompt into the same LLM you used in Scenario 1 (or you may choose to try a different LLM). Request it to generate code based on this detailed prompt.
4. **Execute the Code (Again):** Attempt to execute the code generated using the detailed prompt. Pay close attention to any instructions provided by the LLM regarding setup, libraries, or execution.
5. **Finger Cross**: Do you manage to build a working machine learning model that can make meaningful predictions?

1.1.3 Reflection: LLMs as Tools, Foundational Skills as the Foundation

This tutorial aimed to provide you with a hands-on experience exploring the capabilities and limitations of Large Language Models (LLMs) in a practical coding scenario. As you've likely discovered through Scenarios 1 and 2, while LLMs are undeniably powerful and can generate code snippets with remarkable ease, relying on them *solely*, especially without a foundational understanding of programming, presents significant challenges.

Let's reflect on what you might have encountered, particularly if you approached this as someone new to coding:

Scenario 1: The "Magic" Isn't Always Enough

Your initial, simple prompt likely resulted in code that, at best, was a very basic interpretation of a machine learning task. It may have lacked crucial features, used poor preprocessing, produced weak predictions, or even been non-functional. This illustrates that LLMs aren't mind-readers. They require clear, specific instructions to generate code that aligns with complex requirements. The "magic" is in their ability to process language, but the quality of the output is heavily dependent on the quality of the input and the underlying understanding of the task.

Scenario 2: Detail Improves Output, But Doesn't Solve Everything

The detailed prompt in Scenario 2 undoubtedly led to a more sophisticated and potentially functional workflow. You likely saw improvements in the data handling, model training, and perhaps even evaluation and visual outputs. However, even with a detailed prompt, you might have still faced hurdles. Perhaps the generated code:

- **Contained errors:** LLMs can still produce code with syntax errors or logical flaws, even with detailed instructions.
- **Wasn't fully complete:** The LLM might have missed certain aspects of the desired workflow, or made assumptions that weren't explicitly stated in the prompt.
- **Was difficult to understand and modify:** Even if the code *ran*, you might have found it challenging to decipher *how* it worked, making it difficult to customize or debug.
- **Required external libraries you weren't familiar with:** LLMs might suggest using libraries you've never encountered, adding another layer of complexity to getting the code to run.
- **Presented execution challenges:** Setting up the correct environment, installing libraries, preparing datasets, and running Python code in general can be daunting for beginners.

1.1.4 Common Newbie Experiences Relying Solely on LLMs

If you were approaching this without prior coding experience, you might have encountered frustrations such as:

- `"It doesn't work!"` without understanding *why*: Errors might appear without clear explanations, leaving you unsure how to troubleshoot.
- **Feeling lost in the code:** Even if the code runs, you might not grasp the underlying logic or how different parts interact.
- **Struggling to install libraries or set up the environment:** The practical aspects of running code are often overlooked by simple prompts.
- **Being unable to adapt or extend the code:** If you wanted to add a new feature, try a different model, or fix a bug, you might feel helpless without understanding the code's structure.
- **Over-reliance and eventual disappointment:** Initially impressed by the LLM's output, you might become disillusioned when faced with the practical challenges of making it work and customizing it.

1.1.5 The Crucial Role of Foundational Programming Skills

This experience highlights that while LLMs are powerful *tools* for code generation and can be incredibly helpful for experienced programmers, they are *not* a replacement for fundamental programming knowledge. Think of an LLM like a sophisticated power tool – it can be incredibly efficient in the hands of a skilled crafts person, but potentially dangerous and ineffective for someone without basic carpentry skills.

This course is designed to equip you with those essential ”carpentry” skills for programming. Through the topics we will cover, you will gain:

- **Systematic Thinking and Algorithmic Formulation:** Learn to break down problems into logical steps, enabling you to create effective prompts and understand the underlying logic of code, whether generated by you or an LLM.
- **Python Fundamentals:** Master the basic building blocks of programming in Python, allowing you to read, understand, and write code independently.
- **Control Flow (Repetition and Decision Structures):** Learn how to create dynamic and interactive programs, going beyond simple linear code generation.
- **Functional and Object-Oriented Programming:** Develop modular and reusable code, making your projects more manageable and scalable, and enabling you to better understand more complex LLM-generated code.
- **Data Structures (Arrays) and Data Visualization:** Learn to work with data effectively and present it visually, expanding the scope of projects you can undertake.
- **Error Management and Debugging:** Develop essential skills to identify, diagnose, and fix problems in your code, and in code generated by LLMs, making you a more resilient and independent programmer.
- **Test-Driven Development:** Learn to write robust and reliable code by incorporating testing methodologies.

1.1.6 Course Learning Outcomes Revisited

By mastering these topics, you will directly achieve the course learning outcomes:

- **Apply the process, methods and fundamental knowledge in designing algorithms and the concepts of programming logic:** You'll move beyond simply prompting LLMs and gain the ability to design algorithms yourself, understanding the "why" behind the code.
- **Construct the logic and algorithm design to represent logic in high-level programming language:** You'll be able to translate your algorithmic thinking into actual Python code, whether writing it yourself or effectively guiding an LLM.
- **Demonstrate the ability to gain new knowledge of related entrepreneurship skills and share with others:** A solid programming foundation empowers you to explore entrepreneurial opportunities in technology and effectively communicate your knowledge to others.

> "LLMs are impressive, but they're not a magic bullet. They're tools, and like any tool, they have their limitations."
>
> LogicalBum

Session 2

Variables

Section 2.1 Introduction

Goal

Practice creating and updating Python variables, use basic operators and comparisons on one Iris flower sample, and build the stable variable names that later sessions will reuse.

You will work in

Use the following file for this session:

- `session2/session2_iris_template.py` (main student file)

How to run the file: Always run this script from the project root using the terminal command:
`python3 session2/session2_iris_template.py`
Do not edit other files.

This is **Session 2** of the course. This lab session starts our **one-dataset journey**: the **Iris dataset**. From session 2 until session 12, we will consistently use Iris only, while building programming skills step-by-step until we can build and compare simple classifiers. Programming becomes easier when you are comfortable changing values and observing outputs. Your goal for this session is to practice changing flower measurements and seeing how derived values and comparisons change.

Hands-on file for this session

Work only in the following file:

- `session2/session2_iris_template.py`

How to run the file: Use `python3` in the integrated terminal or click the Run button in VS Code. **Do not edit other files.**

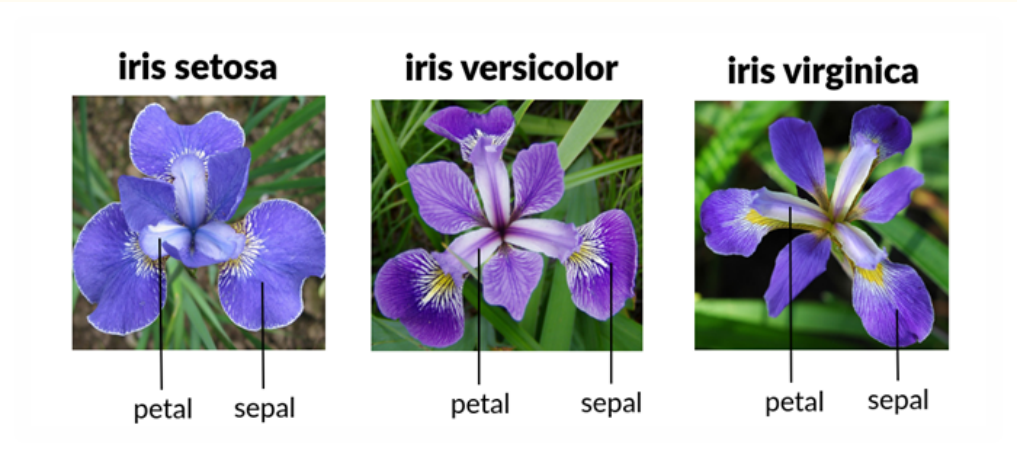

Figure 2.1: Session 2 focus on Iris fundamentals (image extracted from miro.medium.Com).

Section 2.2 What You Will Learn

For today (session 2), we are not doing machine learning yet. Instead, we treat one Iris flower as normal Python variables so you can practice: Upon completing this tutorial, you will be able to:

1. Declare clear variable names for one Iris sample,
2. Compute a derived value with operators,
3. Use a comparison that returns `True`/`False`,
4. Print your student label for submission,
5. Define stable variable names reused in Sessions 3-5.

Section 2.3 Canonical Variables

It is imperative to use consistent variable names across sessions. We enforce these names so next session's template can reuse your variables without changes. Therefore, we will familiarize you with the canonical variable names for Iris samples and rule settings.

- Sample fields: `id`, `sepal_length`, `sepal_width`, `petal_length`, `petal_width`, `species`
- Rule settings: `threshold`, `feature_name`, `positive_label`, `negative_label`, `label_key`

Section 2.4 Iris Dataset Background

The Iris flower dataset (often called Fisher's Iris dataset) is a classic multivariate dataset introduced by the statistician and biologist Ronald Fisher in his 1936 paper on using multiple measurements in taxonomic problems. It contains sepal and petal measurements for three Iris species and underpins many foundational examples in linear discriminant analysis. This material serves as the single dataset we will reuse throughout the course, so a solid grasp of its structure accelerates every subsequent session.

Section 2.5 Flower Features and Data Types

Each Iris flower (and any other object we might study) has observable characteristics such as color, sepal width, petal length, or even the species label itself. In data analysis these characteristics are called **features**. When we write code, each feature is stored inside a **variable**. So when you see a programming variable like `sepal_length`, you can think of it as the code representation of a real-world feature.

Variables are not all the same: each one stores a specific data type. For example, lengths and widths are usually `float` values because they can contain decimals, while labels such as the species name are `string`s. Keeping track of data types helps avoid bugs and makes later calculations easier. The table below lists the canonical variables you will manipulate throughout the Iris exercises along with their typical data types.

Variable / Feature	Data Type	What it represents
`id`	string	Unique identifier for the flower sample
`sepal_length`	float	Sepal length measurement in centimeters
`sepal_width`	float	Sepal width measurement in centimeters
`petal_length`	float	Petal length measurement in centimeters
`petal_width`	float	Petal width measurement in centimeters
`species`	string	Class label such as `setosa` or `virginica`
`feature_vector`	list of floats	Ordered measurements (e.g., [sepal, petal values])
`threshold`	float	Rule cutoff when comparing a measurement
`feature_name`	string	Name of the feature the rule evaluates
`positive_label`	string	Label to assign when the condition is true
`negative_label`	string	Label to assign when the condition is false
`label_key`	string	Dictionary key used to store the chosen label

Table 2.1: Canonical Iris variables and the data types we will use in code.

This provides a structured way to store and access various attributes of the flower, including:

1. **Flower Identity:** A unique identifier for the sample (`id`).
2. **Sepal Measurements:** `sepal_length` and `sepal_width`.
3. **Petal Measurements:** `petal_length` and `petal_width`.
4. **Species Label:** The class name for the sample (`species`).
5. **Feature Vector:** A compact list of numeric values (`feature_vector`) used for quick processing.

Section 2.6 Starter: One flower as variables

Task 1: Print Flower Summary

Run the script `session2/session2_iris_template.py`.

Open the file named `session2/session2_iris_template.py`. Run the script by clicking the Green "Run" button at the top-right corner in VS Code.The Run button looks like a green triangle (similar to a play button in media players) as shown in 2.2. When you click it, VS Code will execute the Python script and display the output in the terminal below.

You should see a printed summary of the flower's ID in the terminal.

You should see the output as below:

```
=== Flower Summary ===
ID: flower1
```

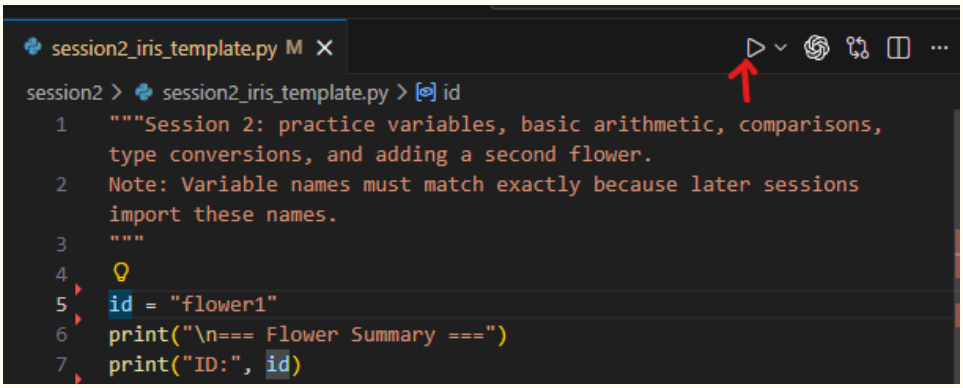

Figure 2.2: Green "Run" button in VS Code to execute the Python script.

The terminal is located at the bottom of your VS Code window. If you don't see it, you can open it by going to the menu and selecting `View > Terminal` or using the shortcut `Ctrl + ` ` (Windows) or `Cmd + ` ` (Mac). The terminal should look like in the image 2.3.

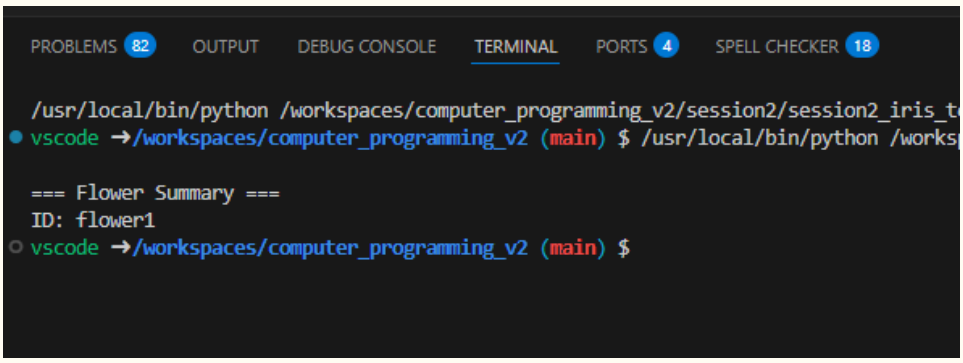

Figure 2.3: Example terminal output after running the starter Iris script.

2.6.1 Experimenting with different Iris values

Task 2: Change this to "flower_2x

Change the value of the `id` variable from `"flower1"` to `"flower_2x"`.

As mentioned above, programming becomes easier when you are comfortable changing values and observing outputs. Your goal for this session is to practice changing flower Id.

First, try changing the flower ID value from `id = "flower1"` to `id = "flower_2x"` in the code. Run the script again and observe the output.

You should see:

```
=== Flower Summary ===
ID: flower_2x
```

Checkpoint

You should see `ID: flower_2x`.

Common Mistakes

If you get an error, check if you accidentally removed the quotes around the string or misspelled the variable name.

Great! You have successfully modified the flower ID variable. Now, just to ensure you are comfortable with changing values, try changing the flower ID to any other string you like (e.g., `"my_flower"`, `"iris_sample_01"`, etc.). Run the script again and observe the output. You should see the updated flower ID in the output as below:

```
=== Flower Summary ===
ID: my_flower
```

Now, change it back to `"flower1"` for the next tasks.

2.6.2 One Iris Flower as Variables

Task 3: Define more variables for one Iris flower

Decomment some of the lines by removing the symbol # at the beginning of line and, change the variable values to represent the measurements and species of the flower as described in the instructions.

Now that you have practiced changing the flower ID, let's add more variables to represent the flower's measurements and species. In the Python template, some of the variables are already defined for you. You just need to fill in the values.

- Let's set the `id` flower to `"flower1"`.
- Then, we set the `sepal_length` equal to `5.1` (float).
- Next, we set the `sepal_width` equal to `3.5` (float).
- Then, we set the `petal_length` equal to `1.4` (float).
- Next, we set the `petal_width` equal to `0.2` (float).
- Finally, we set the `species` equal to `"setosa"` (string).

Then, when you run the script, you should see the full flower summary printed as below:

```
=== Flower Summary ===
ID: flower1
Sepal Length: 5.1
Sepal Width: 3.5
Petal Length: 1.4
Petal Width: 0.2
Species: setosa
```

Checkpoint

You should see the 6 attributes printed cleanly without errors.

Common Mistakes

Check your variable names exactly: `sepal_length` not `sepal_lenght`.

Section 2.7 Derived Variable with Operators

Task 4: Compute petal area

Place the variable petal_length and `petal_width`, but multiply them together to compute the petal area and store it in a new variable named `petal_area`.

As we discussed in the lecture, you can compute new values using existing variables and operators. For example, you can compute the petal area by multiplying `petal_length` and `petal_width`. Add the following line to your code to compute `petal_area`:

```
petal_area = petal_length * petal_width
```

2.7.1 Your Task: Compute Petal Area

Before you run the script, uncomment the print statement for petal area in the flower summary section, or else it will not show in the output. There are two ways to uncomment:

- Remove the `#` symbol at the start of the line that prints petal area.
- Or, if you are using VS Code, you can highlight the line and press `Ctrl + /` (Windows) or `Cmd + /` (Mac) to toggle commenting.

When you run the script again, you should see the petal area printed as below:

```
Petal Area: 0.28000000000000003
```

Checkpoint

You should see a floating-point number representing the area.

Common Mistakes

If nothing prints, ensure you properly uncommented the line and you don't have indentation errors.

Section 2.8 Comparison Result

2.8.1 Introduce Stable Rule Variables

Next session (session 3), we will build a simple rule-based classifier. This mimics simple screening rules used in biology or triage systems! A common beginner rule for Iris is:

If the petal length is less than 2.0 cm, classify the flower as "setosa"; otherwise, classify it as "not_setosa".

In mathematical notation, this can be expressed as:

$$\text{if } petal_length < 2.0 \Rightarrow \text{"setosa"}, \text{ else "not_setosa"}$$

To prepare for this, we will define stable variable names for the rule settings.

Create a threshold variable

1. Create a variable `threshold` with value `2.0`.

2. Compare `petal_length` with `threshold`.

3. Print the boolean result.

Define these values exactly:

```
threshold = 2.0
feature_name = "petal_length"
positive_label = "setosa"
negative_label = "not_setosa"
label_key = "species"
```

These variables are preparation for Session 3 classifier logic.

Try different `petal_length` values and observe how `True/False` changes.

2.8.2 Task 4: Comparison Result (No if-statement yet)

Task 5: Comparing with threshold

Create a boolean variable `is_short_petal` that checks if `petal_length` is less than `threshold`.

Create and print:

```
is_short_petal = petal_length < threshold
```

This gives a boolean output and prepares you for Session 3 conditionals.

Checkpoint

You should see `True` for the setosa sample since $1.4 < 2.0$.

Section 2.9 Take Home Exercise: Add a Second Flower with _2 Suffix

You are tasked to create a second flower based on the *Iris setosa* shown in the image below. Define the variables with the measurements (sepal length, sepal width, petal length, and petal width) exactly as displayed in the figure. To avoid confusion with the variables from the previous tasks, append the suffix `_2` to each variable name (e.g., `sepal_length_2`, `species_2`). This suffixing ensures we can convert them correctly in Session 3.

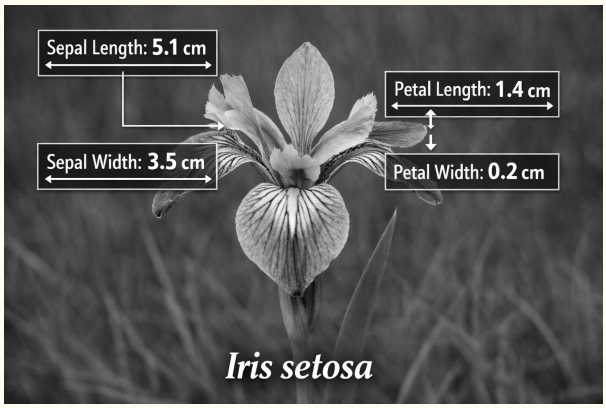

Figure 2.4: Iris setosa flower with sepal length, sepal width, petal length, and petal width with measurement as shown in the image (image artificially generated).

Your specific tasks are:

1. Define the variables: `sepal_length_2`, `sepal_width_2`, `petal_length_2`, `petal_width_2`, and `species_2`.
2. Assign the values visible in the image to these variables.
3. Compute the petal area for this second flower and store it in a variable named `petal_area_2`.
4. Print the calculated area.

```
# Example of the computation (ensure you define the variables first!)
petal_area_2 = petal_length_2 * petal_width_2
print(petal_area_2)
```

Hint

Use the measurements shown in the figure to define `sepal_length_2`, `sepal_width_2`, `petal_length_2`, `petal_width_2`, `species_2`. Make sure the expected output matches what you did for the first flower.

Checkpoint

Ensure you get the right `petal_area_2` by multiplying 1.4 by 0.2.

Lecturer Only Content

Solution Notes for Task 6: *Instructor values:* `sepal_length_2 = 4.9`, `sepal_width_2 = 3.0`, `petal_length_2 = 1.4`, `petal_width_2 = 0.2`, `species_2 = "setosa"`.

Section 2.10 Summary & Review

- **Variables:** You defined variables to store flower features.
- **Data Types:** You converted data between strings and floats.
- **Operators:** You derived new variables using basic arithmetic.
- **Comparisons:** You practiced boolean expressions for rule checks.

Review Questions:

1. What happens if you set `petal_length = 3.0` in the comparison task?
2. Why do we use floats for length but strings for species?
3. What is the difference between `2.0` and `"2.0"` in Python?

Micro-Exercise: Change the `threshold` variable to `1.0` and predict what the `is_short_petal` output will be before running it!

Session 3

Lists, Loops, and Conditionals

Section 3.1 Introduction

Goal

Build a simple rule-based Iris classifier using dictionaries, lists, loops, and conditionals, while keeping the logic sequential so it can be refactored into functions in Session 4.

You will work in

Use the following file for this session:

- `session3/session3_iris_template.py` (main student file)

How to run the file: Always run this script from the project root using the terminal command:
`python3 session3/session3_iris_template.py`
Do not edit other files. We will reuse this file in later sessions.

This session is a continuation of Session 2 (variables and basic data types) and is designed to bridge you into Session 4. In Session 3, we intentionally keep the implementation **sequential** (no functions yet) so that Session 4 can refactor the exact same logic into reusable functions.

The task for Session 3 is to build a small rule-based classifier for the Iris dataset. You will use:

- **lists** to store multiple samples,
- **dictionaries** to store a single sample with named keys,
- a **for loop** to process all samples,
- **if-else** conditionals to make predictions and evaluate them.

Hands-on file for this session

Work only in the following file:

- `session3/session3_iris_template.py`

How to run the file: Use `python3` in the integrated terminal or click the Run button in VS Code. **Do not edit other files.** We will reuse this file in later sessions.

Section 3.2 What You Will Learn

By the end of this tutorial, you will be able to:

1. **Store structured data using dictionaries:** Represent one Iris flower sample using meaningful keys such as `petal_length` and `species`.
2. **Scale up using a list of dictionaries:** Combine many sample dictionaries into a single `dataset` list.

3. **Automate repeated work using loops:** Use one loop to classify and evaluate every sample.
4. **Use conditionals for decision making:** Apply a simple rule based on `petal_length` and compute accuracy.

Section 3.3 Continuity from Session 2 (Rule Variables)

In Session 2 you introduced a simple rule with named variables. We keep those variables here **unchanged** because Session 4 expects the same naming and behavior.

The rule is:

> If `petal_length` is less than `threshold`, predict `setosa`; otherwise predict `not_setosa`.

Keep the following values exactly as they are in the session 3 template:

```
sepal_length = 5.1
sepal_width = 3.5
petal_length = 1.4
petal_width = 0.2
species = "setosa"
```

Listing 1: Example Iris dataset variables

Section 3.4 Python Lists

A list in Python is a versatile data structure used to store ordered collections of items. Lists are ordered, mutable (you can change their contents). List can contain any type of data, such as numbers, strings, or even other lists (or dictionaries, as we'll see later).

Here's an example of a list containing numerical values, representing petal lengths:

code:IrisPetalLengths

```
iris_petal_lengths = [
    1.4, 10.1, 1.7, 1.5,
    4.7, 4.5, 6.0, 5.1,
]
```

3.4.1 Accessing items in a list:

Python lists are indexed starting from zero. You access individual items using square brackets:

```
first_petal_length = iris_petal_lengths[0]   # 1.4
second_petal_length = iris_petal_lengths[1]  # 10.1
```

3.4.2 Changing list items:

Lists are mutable, meaning you can modify their values. For example, to change the first petal length:

```
iris_petal_lengths[0] = 1.5  # Changes the first petal length to 1.5
```

3.4.3 Adding items to a list:

You can append new values to the list easily. For example, to add a new petal length,

```
iris_petal_lengths.append(4.0)  # Adds a new value at the end
```

Section 3.5 Concept: Dictionaries (One Iris Sample)

Introduction to Dictionaries

Dictionaries are a fundamental data structure in Python used for storing collections of **key-value pairs**. Unlike lists, which are ordered sequences accessed by numerical index, dictionaries are unordered collections where you access data using **keys**. Think of a real-world dictionary: you look up a word (the key) to find its definition (the value).

Dictionaries are incredibly useful when you need to associate pieces of information with specific labels or identifiers, making your code more readable and maintainable.

In our previous sessions, we've seen how to store individual pieces of information using separate variables. While this works for a small amount of data, it quickly becomes cumbersome when dealing with many flowers or features.

Using a dictionary, we can group related pieces of information under meaningful names. For example, we can convert the code variables (as in block 1) into a single dictionary that represents one flower sample as shown in block 2.

```
sample = {
    "id": "flower1",
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2,
    "species": "setosa"
}
```

Listing 2: Example Iris dataset variables

In this example, "id", "sepal_length", "sepal_width", "petal_length", "petal_width", and "species" are keys, and their corresponding values are the data associated with each key. Notice that values can be of different types, including strings, nested dictionaries, and lists.

Lecturer Only Content

Ask student, about which is string, which is number, and which is a label. This is a good opportunity to reinforce the concept of data types in the context of dictionaries.

3.5.1 Creating Dictionaries

There are several ways to define a dictionary in Python. Let's explore three common methods:

- **Literal syntax (`{}`):** This is the most direct and common way to create dictionaries.
- **Using `dict([])` with tuples:** This method involves passing a list of tuples to the `dict()` constructor.
- **Using `dict()` with keyword arguments:** This approach uses the `dict()` constructor and assigns key-value pairs using keyword-like syntax.

Literal Syntax `{}` This is the most straightforward way to create a dictionary. You enclose key-value pairs within curly braces `{}`. Each key-value pair is separated by a colon `:`, and pairs are separated by commas `,`:

```
flower_parts = {
    "id": "flower1",
```

```
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2,
    "species": "setosa"
}
```

`dict([])` with Tuples You can also create a dictionary by passing a list of tuples to the `dict()` constructor:

```
flower_parts = dict([
    ("id", "flower1"),
    ("sepal_length", 5.1),
    ("sepal_width", 3.5),
    ("petal_length", 1.4),
    ("petal_width", 0.2),
    ("species", "setosa")
])
```

This is useful when your data is already structured as key-value pairs in tuples.

`dict()` with Keyword Arguments The `dict()` function can also be used with keyword arguments. This is often more readable when creating dictionaries with string keys:

```
flower_sample1 = dict(
    id="flower1",
    sepal_length=5.1,
    sepal_width=3.5,
    petal_length=1.4,
    petal_width=0.2,
    species="setosa"
)
```

Regardless of the method you choose, the variable now holds a dictionary representing a flower's measurements.

Canonical Keys are Important!

Before you start the task, it is critical to use these exact keys: `id`, `sepal_length`, `sepal_width`, `petal_length`, `petal_width`, and `species`. Later code (in Session 4 and beyond) relies on these canonical names for dictionary lookups. If you misspell them, the code will fail to find the values!

3.5.2 Your Task: Replacing sequential variable declaration with dict

Task 1: Create A dictionary for second flower

... but use the same values from Session 2 (the setosa image) and remove the `_2` suffix from the keys.

Open the file `session3/session3_iris_template.py`. We will now convert the variables you created in Session 2 into a dictionary format. For the second flower (`flower2`), remove the `_2` suffix from the keys (e.g., use `sepal_length` instead of `sepal_length_2`), but keep the numerical values from the Session 2 setosa image.

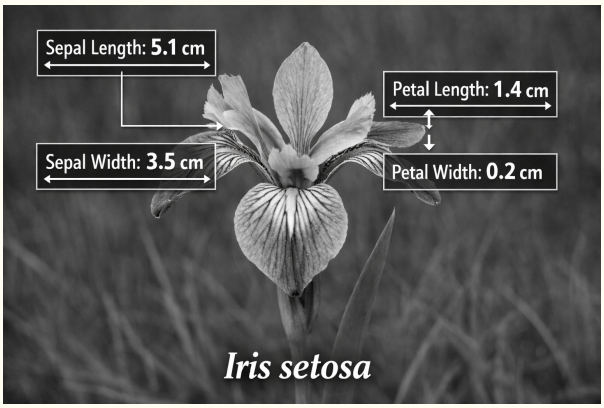

Figure 3.1: Iris setosa flower measurements (recap from Session 2).

Hint

Use the same measurements from Session 2 but without the `_2` suffix when building the dictionary for `flower2`.

Checkpoint

You should have two dictionaries defined: `flower1` and `flower2`, using literal syntax `{}`.

Common Mistakes

Make sure you didn't forget commas at the end of each key-value pair, and ensure the keys have quotes around them (e.g., `"sepal_length"`).

3.5.3 Accessing Data from Dictionaries

You access values in a dictionary using their corresponding **keys** within square brackets `[]`.

```
print(flower_sample1['id']) # Print the id of the flower
```

Lecturer Only Content

Solution: *The student should create a dictionary for the second flower using the values from the image:*

```
flower2 = {
    "id": "flower2",
    "sepal_length": 4.9,
    "sepal_width": 3.0,
    "petal_length": 1.4,
    "petal_width": 0.2,
    "species": "setosa"
}
```

Section 3.6 List of Dictionaries

Task 2: Create list of dictionaries

Combine your two flower dictionaries into a list called `dataset`.

One Iris sample is useful, but the real task is to process **many** samples. We therefore store multiple dictionaries inside a list called `dataset`.

Start from this pattern (your template already contains a filled example for flower 1, but you'll combine them):

```
dataset = [flower1, flower2]
```

Checkpoint

Ensure `dataset` is a list containing your two dictionary variables.

Section 3.7 For Loop

A **for loop** lets you run the *same* block of code multiple times without copying and pasting it. It is especially useful when you want to process **each item in a sequence** (for example: a list of numbers, a list of strings, or a dataset).

3.7.1 The Core Idea: "Do this for each item"

Think of a for loop as reading a list one element at a time:

> "For each item in this collection, run these instructions."

Most for loops involve a **loop variable** (a temporary name you choose) that takes on a new value each iteration.

For example, conceptually iterating from 1 to n looks like:

$$\text{for } i = 1 \text{ to } n \text{ do: execute the loop body.}$$

Here, i is the loop variable and it changes as the loop runs.

3.7.2 What Parts Make Up a Loop?

In many languages, a for loop can be described using four pieces:

- **Initialization:** where the loop starts (e.g., $i = 1$).
- **Condition:** when the loop should stop (e.g., keep going while $i \leq n$).
- **Update:** how the loop variable changes each time (e.g., $i \leftarrow i + 1$).
- **Body:** the code that repeats each iteration.

In Python, you most often write for loops in the simpler "for each item in this sequence" style, which avoids manually writing the condition and update step.

3.7.3 Looping Over the Dataset

Task 3: Create a for loop to process the dataset

Write a for loop that processes each sample in the `dataset` list. Inside the loop, print the sample's `id`, `petal_length`, and `species`.

When you have a dataset stored as a **list of dictionaries**, one of the cleanest ways to process every sample is a `for` loop. The loop below assigns each dictionary to the variable `sample`, one at a time:

```
for sample in dataset:
    print(sample["id"], sample["petal_length"], sample["species"])
```

How to Read This Code

- `dataset` is the full collection (a list).
- `sample` is one element from that list (a dictionary).
- On each iteration, `sample` refers to a *different* dictionary.

Interesting fact

The `print` step is a good beginner-friendly debugging habit:

- It confirms the keys (like `"id"` and `"petal_length"`) actually exist.
- It lets you check that values look reasonable before adding more logic.
-

Once you trust the data shape and values, you can replace the `print` line with classification or processing code.

Section 3.8 Conditionals

3.8.1 Conditionals for Classification

Task 4: Use an if-else statement to classify each sample

A **conditional** lets your program **make a decision** based on data. In many application, the most common form is an **if–else** statement:

- if a condition is `True`, run the indented "if" block;
- otherwise, run the indented "else" block.

Using a Rule to Predict a Label

In our classifier, the decision rule compares **one feature value** to a **threshold**. The variable `feature_name` tells us which feature to look up in the current `sample` dictionary:

```
# Example rule: compare the chosen feature to a threshold
if sample[feature_name] < threshold:
    y_pred = positive_label
else:
    y_pred = negative_label
```

How to Read This Code

- `sample[feature_name]` retrieves the feature value (e.g., `sample["petal_length"]`).
- `< threshold` checks whether that value is below the cutoff.
- `y_pred` stores the predicted label for this sample.

Common Beginner Notes

- **Indentation matters:** the indented lines belong to the `if` or `else` block.
- **Exactly one branch runs:** either the `if` block or the `else` block, never both.
- If you want "less than or equal," use `<=` instead of `<`.

3.8.2 Task 1: Initialize Metrics and Predictions List

Before the loop, initialize counters and a list for predictions:

```
correct = 0
wrong = 0
total = 0
y_pred_list = []
```

3.8.3 Task 2: Classification Rule

Inside the loop, compute `y_pred` using the Session 2 rule. Do **not** define any functions yet.

3.8.4 Task 3: Convert the True Label and Update Metrics

Inside the same loop:

1. Convert the true label from `sample[label_key]` to `y_true` (if it is `positive_label`, keep it; otherwise `negative_label`).
2. Compare `y_pred` and `y_true`.
3. Update `correct`, `wrong`, and **always** update `total`.
4. Append `y_pred` into `y_pred_list`.

Checkpoint

Your loop should be processing each sample. **Self-check:** Does `total` increase for every sample processed? Even if the prediction is wrong, `total` must increase by 1.

Common Mistakes

Forgetting to increment `total` inside the loop, or accidentally only incrementing it if the prediction was correct.

3.8.5 Task 4: Print Per-Sample Trace Lines (Required Format)

For continuity with Session 4, print one line per sample exactly using this format inside the loop:

```
print(
    f"id={sample['id']} | true={y_true} | pred={y_pred} | "
    f"petal_length={sample['petal_length']}"
)
```

3.8.6 Task 5: Final Summary (After the Loop)

After processing all samples, compute and print accuracy:

```
accuracy = (correct / total) * 100 if total > 0 else 0.0
```

Checkpoint

You should see output similar to this at the end:

```
Correct: 2
Wrong: 0
Total: 2
Accuracy (%): 100.0
All predictions: ['setosa', 'setosa']
```

Section 3.9 Summary & Review

- **Dictionaries:** You learned to store one object's features using named keys.
- **Lists:** You used lists to hold multiple dictionary items.
- **Loops:** You iterated over the dataset using a `for` loop.
- **Conditionals:** You used `if/else` to classify samples.
- **Metrics:** You tracked accuracy by counting correct predictions over total samples.

Reflection Questions:

1. Why is a list of dictionaries better than having many individual variables (like `sepal_length_2`, `sepal_length_3`)?
2. What happens to your dictionary lookups if you misspell a key?

Micro-Exercise: Add a third sample to your `dataset` (e.g., make up a flower with `petal_length: 4.5` and `species: "versicolor"`). Run the script again and observe how the loop, trace lines, and accuracy automatically update!

Session 4

Functions

Section 4.1 Introduction

Goal

Refactor sequential code into reusable functions, load a CSV dataset, and preserve the classifier rule from Session 3. This builds a clean foundation for upcoming sessions.

You will work in

Use the following files for this session:

- `session4/session4_iris_template.py` (student template)
- `session4/iris_data.csv` (dataset)

How to run the file: Use the terminal command:
`python3 session4/session4_iris_template.py`
Do not edit other files. We will reuse this file in later sessions.

This session is all about how functions can dramatically improve the way you structure your code, making it more organized and reusable. Think of **functions** as essential building blocks for any programmer. They allow you to break down complex projects into smaller, more manageable, and importantly, reusable parts. By learning to effectively use functions, you'll be writing Python code that is not only cleaner and more efficient but also significantly easier to understand and maintain.

In Session 3, you built a solid foundation by learning about lists, loops for repetitive tasks, and conditional statements for decision-making. Now, we're going to elevate your Python skills further. This session, we'll focus on structuring your Python scripts more effectively using functions. We'll revisit code from a previous tutorial that was written sequentially and transform it by incorporating functions.

Section 4.2 What You Will Learn

In this session, you will learn to:

1. **Refactor Code into Functions**: Take existing, sequential scripts and reorganize them using functions. This will teach you how to identify blocks of code that can be encapsulated into reusable functions.

2. **Understand Functions**: Learn the fundamental concept of functions – what they are, how they are structured in Python, and why they are so beneficial for writing efficient and organized code.

3. **Load a Dataset from CSV**: In real projects, datasets can come from many file formats. For now, we use `.csv` because it is simple, human-readable, and supported by Python without extra libraries.

4.2.1 Functions Primer

A **function** is a named block of code designed to perform one specific job. Instead of writing the same lines repeatedly, you define the logic once and *call* the function whenever you need it. Functions take *arguments* (inputs), perform actions, and *return* a value (output).

```
def greet_student(name):
    # This function takes a name and returns a greeting
    return "Hello, " + name

print(greet_student("Alice"))
```

4.2.2 What is the common rule for creating functions?

There are several rules of thumb for creating functions, but a key one is: **Each function should do one thing and do it well.** This means that a function should have a single responsibility or task. If you find yourself writing a function that is trying to accomplish multiple tasks, it's often a sign that you should break it down into smaller, more focused functions. This makes your code easier to read, test, and maintain. For example, if you have a function that both processes data and prints results, consider splitting it into two functions: one for processing and another for printing.

4.2.3 The Benefit of function

Using functions has several benefits:

- **Reusability**: Once a function is defined, you can call it multiple times throughout your code without having to rewrite the same logic.
- **Organization**: Functions help break down complex problems into smaller, more manageable pieces, making your code easier to read and understand.
- **Maintainability**: If you need to change the logic, you only need to update it in one place (the function definition) rather than in every place where the logic is used.
- **Testing**: Functions can be tested independently, which makes it easier to identify and fix bugs.

Section 4.3 Session 3 to Session 4 Refactor Map

Session 3 sequential blocks become these functions in Session 4:

- session 3 rule block → `determine_binary_label(sample, settings)`
- session 3 truth conversion block → `determine_true_binary_label(sample, settings)`
- session 3 metric update block → `update_metrics(metrics, y_pred, y_true)`
- session 3 compare prediction to truth → `evaluate_prediction(y_pred, y_true)`
- session 3 per-sample classifier step → `classify_sample(sample, settings)`
- session 3 full loop and counters → `initialize_predictions(dataset, settings)`
- session 3 top-level script setup → `setup_application(filepath)` and `main()`

Section 4.4 Canonical Function Set

Complete these functions in `session4_iris_template.py`. The table below shows the mapping:

Table 4.1: Session 4 canonical functions: what to implement and which session 3 block they come from (dataset loaded from CSV).

Function	Code to put inside the function (from Session 3)	Notes
`make_print_status(status_text)`	`print(f"[STATUS] {status_text}")`	Optional helper for readable progress logs.

`determine_binary_label(sample, settings)`	`if sample[settings["feature_name"]] < settings["threshold"]:`
	`  return settings["positive_label"]`
	`return settings["negative_label"]`
		(binary: setosa vs not_setosa).

`determine_true_binary_label(sample, settings)`	`if sample[settings["label_key"]] == settings["positive_label"]:`
	`  return settings["positive_label"]`
	`return settings["negative_label"]`
		Converts species labels into binary truth labels.

`evaluate_prediction(y_pred, y_true)`	`return (y_pred == y_true)`	Returns a boolean correctness flag.

`update_metrics(metrics, y_pred, y_true)`	`if y_pred == y_true:`
	`  metrics["correct"] += 1`
	`else:`
	`  metrics["wrong"] += 1`
	`metrics["total"] += 1`
	`metrics["y_pred_list"].append(y_pred)`
		Refactors the counter update + prediction list append block.

`classify_sample(sample, settings)`	`return determine_binary_label(sample, settings)`	Keeps the main loop clean and prepares for Session 5/6 changes.

`load_iris_data(filepath)`	`dataset = []`
	`with open(filepath, "r", encoding="utf-8") as f:`
	`  reader = csv.DictReader(f)`
	`  for row in reader:`
	`  dataset.append({...})`
		Remark: this session we load the dataset from `CSV` (not JSON). Convert numeric fields to `float`.

Continued on next page

Table 4.1 – continued from previous page

Function	Code to put inside the function (from Session 3)	Notes

`get_default_settings()`	`return {`
	`  "threshold": 2.0, "feature_name": "petal_length",`
	`  "positive_label": "setosa", "negative_label": "not_setosa",`
	`  "label_key": "species"`
	`}`
		Keeps Session 2/3 naming stable; no external settings file yet.

`initialize_predictions(dataset, settings)`	`for sample in dataset:`
	`  y_pred = classify_sample(sample, settings)`
	`  y_true = determine_true_binary_label(sample, settings)`
	`  update_metrics(metrics, y_pred, y_true)`
	`  print(f"id=... | true=... | pred=... | petal_length=...")`
		This is the session 3 loop, but decomposed into functions.

`setup_application(filepath)`	`dataset = load_iris_data(filepath)`
	`settings = get_default_settings()`
	`result = initialize_predictions(dataset, settings)`
		Small pipeline wrapper for readability.

`main()`	`print("Correct:", result["correct"])`	Runs the whole program and prints the final summary.

Section 4.5 Dataset

In typical machine learning work, you may load datasets from many formats.

4.5.1 Task 1: Load the Dataset from CSV

In this course, we start with `CSV` because it is easy to inspect and easy to parse.

Remark (important for this session): the provided `session4/iris_data.csv` currently contains **only Iris setosa** samples. That means the **true label** (after binary conversion) will always be `setosa` for every row. You are still practicing a **binary classifier** (`setosa` vs `not_setosa`) using the same session 3 rule. In later sessions, we will re-introduce `versicolor` and `virginica`.

Load the dataset using `csv.DictReader`:

```
import csv

dataset = []
with open("session4/iris_data.csv", "r", encoding="utf-8", newline="") as csv_file:
    reader = csv.DictReader(csv_file)
    for row in reader:
        dataset.append(
            {
                "id": row["id"],
                "sepal_length": float(row["sepal_length"]),
                "sepal_width": float(row["sepal_width"]),
                "petal_length": float(row["petal_length"]),
                "petal_width": float(row["petal_width"]),
                "species": row["species"],
            }
        )
```

Notice: values read from CSV start as strings, so we convert numeric columns using `float(...)`.

Checkpoint

Your `load_iris_data` function should return a list of dictionaries with length 50.

Common Mistakes

FileNotFoundError: Check if you are running the script from the correct directory (the repository root).
String Type Errors: If you forget `float()`, calculations will fail later because Python cannot multiply strings.

4.5.2 Task 2: Keep the Session 3 Rule Unchanged

Inside your `determine_binary_label` function, keep the same decision rule:

```
if sample[settings["feature_name"]] < settings["threshold"]:
    y_pred = settings["positive_label"]
else:
    y_pred = settings["negative_label"]
return y_pred
```

Canonical Names: Notice how we use `settings["feature_name"]`. We enforce these exact names so the next session's template can import your work without changes.

Section 4.6 Task 2.1: Convert the True Label into a Binary Label

In session 3, you used this block to convert a 3-class species into a binary truth label. Refactor that logic into the function: `determine_true_binary_label(sample, settings)`.

```
if sample[settings["label_key"]] == settings["positive_label"]:
    return settings["positive_label"]
return settings["negative_label"]
```

Section 4.7 Task 2.2: Update Metrics and Collect Predictions

In session 3, you updated counters and stored predictions. In Session 4, refactor that logic into a function: `update_metrics(metrics, y_pred, y_true)`.

Checkpoint

Ensure your `update_metrics` function updates `metrics["correct"]` or `metrics["wrong"]`, appends to `y_pred_list`, and **always** updates `metrics["total"]`.

Common Mistakes

Logic Bug: Only incrementing `total` if the prediction was correct. Check your indentation to ensure `total` increases for every sample!

Section 4.8 Task 3: Maintain Output Compatibility

Inside `initialize_predictions`, use this per-sample output line (same as Session 3):

```
print(
    f"id={sample['id']} | true={y_true} | pred={y_pred} | "
    f"petal_length={sample['petal_length']}"
)
```

Print final summary metrics in `main()`: `Correct`, `Wrong`, `Total`, `Accuracy (%)`, `All predictions`.

4.8.1 Name Label (Required)

Include this in `main()`:

```
print("=== Student Label ===")
```

Section 4.9 About `__main__`

At the bottom of your Python file, you will see:

```
if __name__ == "__main__":
    main()
```

Meaning:

- When you run the file directly, Python sets `__name__` to `"__main__"` and `main()` executes.
- When the file is imported by another file, `main()` does not automatically run.

Section 4.10 Review and Next Steps

Summary:

- **Functions** allow you to bundle code into reusable blocks.
- **CSV Loading** allows you to import external dataset files.
- **Modular Design** sets you up to import these same functions in Session 5.

Review Questions:

1. What is the purpose of `determine_true_binary_label`?
2. Why must we convert numeric values from a CSV file using `float()`?
3. Why do we enforce exact keys like `feature_name` inside the `settings` dictionary?

Micro-Exercise: Change the `threshold` value in `get_default_settings()` and run the file. Observe how the accuracy changes!

Real-World Hook

A rule-based classifier built with modular functions is exactly how basic medical triage rules or spam filters are initially designed before machine learning models are introduced!

Session 5

Positional and Keyword Arguments

Section 5.1 Introduction

Goal

Refactor your Session 4 functions to accept optional parameters, understand the difference between parameters and arguments, and learn how to run the same pipeline in multiple modes.

You will work in

Use the following files for this session:

- `session5/session5_iris_template.py` (main student file)
- `session5/iris_data.csv` (dataset)
- `session5/helper/iris_utils.py` (provided helper module)

How to run the file: Always run this script from the project root using the terminal command:
`python3 session5/session5_iris_template.py`
Do not edit the helper module (`iris_utils.py`).

In the previous sessions, you wrote several **functions** and called them from a `main()` function. That already gave you a powerful skill: you can break a larger program into smaller, reusable pieces.

In Session 5 we keep that same idea, but we make your functions **more flexible** by controlling them through **arguments** when you call them.

Why this matters (Real-World Hook)

Parameterizing functions is akin to how APIs expose configuration options (e.g., passing options to a machine-learning model). It allows users to control the behavior of your code without having to rewrite its internal logic.

Parameters vs. arguments (a common beginner confusion).

- A **parameter** is the name written in the function definition (the "slot" the function expects).
- An **argument** is the actual value you provide when calling the function (what gets put into the slot).

Example:

```
def make_print_status(status_text):
    print(f"[STATUS] {status_text}")

make_print_status("Build dataset")                    # "Build dataset" is an argument
make_print_status("Training model")                   # "Training model" is an argument
```

In this session's tutorial, we focus on **two** ways of providing arguments:

- **Positional arguments:** values are matched to parameters by *order*.
- **Keyword arguments:** values are matched to parameters by *name*.

Understanding these techniques matters because they directly affect:

- how easy your code is to read (especially for someone else),
- how easy it is to change your code later (refactoring without breaking calls),
- how likely you are to introduce subtle bugs (wrong value in the wrong slot).

Section 5.2 What You Will Learn:

Upon completing this tutorial, you'll be able to:

- **Positional arguments:** How Python matches arguments to parameters by position, and why the order matters.
- **Keyword arguments:** How naming parameters in a call improves readability and lets you override only what you need.
- **Default values:** How "optional" parameters work, how `None` is commonly used as a "not provided" marker, and how defaults interact with positional and keyword calls.
- **Beginner-safe debugging:** How to recognize and fix common argument errors such as "missing required argument" or "multiple values for argument".

5.2.1 Important framing:

Session 5 does not replace the session 4 classifier. Instead, it extends the same function set so you can *control the behavior* through function arguments instead of rewriting the classifier logic each time you want a small change. This is a key refactoring mindset: **keep the core rule stable, make the interface flexible**.

1. keep session 4 function names and core classifier behavior unchanged,
2. add positional parameters so a caller can override key values *in order*,
3. add keyword parameters with sensible defaults so a caller can override values *by name*,
4. learn how to import a helper function from another file (your first step toward multi-file projects),
5. call your pipeline like a small API from `main()` to demonstrate different configurations clearly.

If you achieve these goals, you will have a program that can run the same pipeline in multiple modes without duplicating code.

5.2.2 Hands-on Files for This Session

Use:

- `session5/session5_iris_template.py` (your main working file for this session)
- `session5/iris_data.csv` (the dataset you will load and classify)
- `session5/helper/iris_utils.py` (provided helper: a simple "fit" function for learning a threshold)

What you are starting with: a mostly-complete session 5 template where the core classifier functions already exist. **What you will modify:** function signatures and call sites so the pipeline can be configured using arguments.

Section 5.3 Session 4 vs Session 5 (Function-by-Function Changes)

Session 5 is a refactor focused on **interfaces**, not new logic. Keep the same core classifier rule.

5.3.1 Canonical Function Set (Session 5)

Complete these functions in `session5_iris_template.py`. The table below shows the mapping:

Section 5.4 Positional Arguments

Positional arguments are the most common way to pass values to functions. You've already been using them without explicitly realizing it since session 3 of our tutorial session.

In positional arguments, the order of the arguments provided in the function call directly corresponds to the order defined in the function signature.

5.4.1 Loading Iris Data with a Single Positional Argument

Take for example when we want to print a status message using the function `make_print_status`, which is defined to accept a single positional argument `status_text`. When we call this function, we provide a string argument that describes the current status of our program.We are utilizing what is known as a positional argument, where the value we pass is matched to the parameter based on its position in the function definition. In this case, the string we provide will be assigned to the `status_text` parameter when the function executes.

```
make_print_status("Build dataset")

```

The corresponding function definition is:

```
def make_print_status(status_text):
    """Print a small status message.

    Args:
        status_text (str): A short message to show what the program is doing.
    """
    print(f"[STATUS] {status_text}")
```

Here, the string "Build dataset" is passed as a positional argument to the `make_print_status` function. The function then uses this argument to print a status message indicating that the dataset is being built.

5.4.2 Initializing initialize predictions with Two Positional Arguments

Another example is the function `calculate_accuracy`, defined with two positional parameters: `correct` and `total`. When invoking this function, you must ensure that the arguments are provided in the correct sequence. Swapping these arguments inadvertently will cause unexpected behavior or errors in your program.

Thus, clear parameter naming and detailed documentation are essential to prevent such issues.

```
accuracy = calculate_accuracy(correct, total)

```

```
def calculate_accuracy(correct, total):
    """Calculate accuracy percentage.
```

```

    Args:
        correct (int): Number of correct predictions.
        total (int): Number of processed samples.

    Returns:
        float: Accuracy percentage.
    """
    if total > 0:
        accuracy = (correct / total) * 100
    else:
        accuracy = 0.0

    return accuracy

```

5.4.3 Update metrics Using Multiple Positional Arguments

Let's examine a function that requires multiple positional arguments, `update_metrics`, which is responsible for rendering robots on the canvas:

```
print_summary(correct, wrong, total, y_pred_list, accuracy)
```

```
def print_summary(correct, wrong, total, y_pred_list, accuracy):
    """Print the final results after the loop is finished.

    Args:
        correct (int): Number of correct predictions.
        wrong (int): Number of wrong predictions.
        total (int): Number of processed samples.
        y_pred_list (list): List of predicted labels.
        accuracy (float): Accuracy percentage.
    """
    print("\n=== session 4 Summary ===")
    print("Correct:", correct)
    print("Wrong:", wrong)
    print("Total:", total)
    print("Accuracy (%):", round(accuracy, 2))
    print("All predictions:", y_pred_list)
```

This function is built to accept FIVE positional arguments (e.g., `correct`, `wrong`, `total`, `y_pred_list`, `accuracy`). When calling this function, you must provide all five arguments in the exact order they are defined in the function signature. For instance, if you accidentally swap the `correct` and `wrong` arguments, the output will be misleading and incorrect, as the function will interpret the values based on their position rather than their meaning. This highlights the importance of being cautious when using multiple positional arguments, as it can lead to confusion and errors if not handled carefully. Clear documentation and consistent naming conventions can help mitigate these risks, but it's crucial to always double-check the order of arguments when calling functions that rely on positional arguments.

Quiz

Can you identify which other functions in our session 4 codebase also use multiple positional arguments?

5.4.4 Best Practices: Enhancing Code Clarity with Descriptive Names and Docstrings

In scenarios with multiple positional arguments (2 or more), the significance of clear `docstring` and clear naming variables becomes even more importance. These practices help prevent confusion and

errors, ensuring that anyone using the function (e.g., update_metrics), understands what each argument represents and in what order they should be provided.

Yet, it's important to recognize that mistakes can still occur. Users might inadvertently supply incorrect input or mix up the order of arguments. So, I hope this highlight the important ensuring supplying the right value and order for correct operation of the function's operation.

5.4.5 Self Reflection: Your Experience with Positional Arguments

Nevertheless, either you realise it or not, but you actually have been working with the concept of positional arguments since the third session of our tutorials. So Kudos to you for that. Therefore, I did not design any specific task for this section, as I believe you already have a good grasp of how positional arguments work in `Python` functions.

Section 5.5 Keywords Arguments

Keyword arguments offer an alternative way to provide values to function parameters explicitly by their names, regardless of the order. Unlike positional arguments, keyword arguments allow you to specify exactly which parameter is being assigned a value, enhancing readability and reducing errors.

Advantages of Keyword Arguments:

There are several advantages:

- Enhanced readability and clarity in function calls.
- Flexible argument ordering, reducing potential errors.
- Easy handling of optional parameters with default values.

Keyword Argument Syntax

When you use keyword arguments, you specify the name of the parameter followed by an equal sign and the value it should receive within the function call. For example,

width=1000

The benefit of this approach is that you can rearrange the order of arguments when calling the function without affecting its behavior, as long as all required arguments are provided.

So far, we have not been using keyword arguments in our code, but we will start introducing them in this session to give you more control over the behavior of your functions without having to change their internal logic.

5.5.1 Two keyword argument

While seems overkill to using a keyword argument with only two parameter, but for the sake of learning, we will just do it.

```
def calculate_accuracy(correct, total):
    """Calculate accuracy percentage.

    Args:
        correct (int): Number of correct predictions.
        total (int): Number of processed samples.

    Returns:
```

```
        float: Accuracy percentage.
    """
    if total > 0:
        accuracy = (correct / total) * 100
    else:
        accuracy = 0.0

    return accuracy

```

and we can refactor it to have two keyword arguments like this:

```

def calculate_accuracy(correct=0, total=0):
        """Calculate accuracy percentage.

    Args:
        correct (int): Number of correct predictions.
        total (int): Number of processed samples.

    Returns:
        float: Accuracy percentage.
    """
    if total > 0:
        accuracy = (correct / total) * 100
    else:
        accuracy = 0.0

    return accuracy

```

The advantage of this approach now is that, you can swap the order of the arguments when calling the function without affecting its behavior, as long as all required arguments are provided.

For example, you can call `calculate_accuracy(total=total, correct=correct)` and it will work just fine.

Your Task:

Task 1: Create a two keyword argument function

refactor the `calculate_accuracy` function to have two keyword arguments

In the code template, refactor the `calculate_accuracy` function to have two keyword arguments: `total` and `correct`. Then, update all call sites of this function to use keyword arguments instead of positional arguments.

Also, you need to update, in the function calling , to put the keyword arguments in the correct order, or you can swap the order of the arguments when calling the function without affecting its behavior, as long as all required arguments are provided. For example, you can call `calculate_accuracy(total=total, correct=correct)` and it will work just fine.

5.5.2 Many many keyword

It is also possible to have many keyword arguments in a function. For example, in the `update_result_counts` function, you can have multiple keyword arguments to control different aspects of Updating the counters and prediction list for one sample.

Recall that in previous session the setup application function accept 6 positional arguments, which means the order of the arguments matters. This can be inconvenient and error-prone.

```python
def update_result_counts(correct, wrong, total, y_pred_list, y_pred, y_true):
    """Update the counters and prediction list for one sample.

    Args:
        correct (int): Current number of correct predictions.
        wrong (int): Current number of wrong predictions.
        total (int): Current number of processed samples.
        y_pred_list (list): Current list of predictions.
        y_pred (str): Predicted label.
        y_true (str): True label.

    Returns:
        tuple: Updated correct, wrong, total, y_pred_list
    """
    if y_pred == y_true:
        correct += 1
    else:
        wrong += 1

    total += 1
    y_pred_list.append(y_pred)

    return correct, wrong, total, y_pred_list
```

With keyword arguments, you can specify only the parameters you want to override, and the rest will use their default values.

For instance, below are incomplete example, but given your experience in task 1, I expect you can do it, but you can refactor it as.

```python
def update_result_counts(correct=0, wrong, total, y_pred_list, y_pred, y_true):
    """Update the counters and prediction list for one sample.

    Args:
        correct (int): Current number of correct predictions.
        wrong (int): Current number of wrong predictions.
        total (int): Current number of processed samples.
        y_pred_list (list): Current list of predictions.
        y_pred (str): Predicted label.
        y_true (str): True label.

    Returns:
        tuple: Updated correct, wrong, total, y_pred_list
    """
    if y_pred == y_true:
        correct += 1
    else:
        wrong += 1

    total += 1
    y_pred_list.append(y_pred)

    return correct, wrong, total, y_pred_list
```

Now that you have refactored, try to refactor the others as well. Based on the docstring,which expect an integer, we set the default value of `correct` to 0. Similarly, we can set default values for the other parameters as well, such as `wrong` and `total` equal to 0. However, if you notice, the `y_pred_list` is a list, and it is not a good idea to set it equal to zero.

Similarly, for the `y_pred` and `y_true` parameters, which are expected to be strings, it is not appropriate to set their default values to zero either. So,we will set it to None, which is a common practice in Python to indicate that a parameter is optional and does not have a meaningful default value.

Your Task:Update

now in the code, refactor the `setup_application` function to have multiple keyword arguments: `filepath`, `threshold`, `print_each`, and `use_fit`. Then, update the function logic to apply the fit or threshold overrides based on the provided arguments.

5.5.3 Adding Default Values with Keyword Arguments

So far, the function `load_robot_data` has been using positional arguments. However, we can improve its flexibility by using keyword arguments. For example, we can refactor the function to have a keyword argument for the threshold value, if the user does not provide a threshold, it will use a default value. This way, the function can be called with or without the threshold argument, and it will still work correctly.

```
def setup_application(filepath=None, threshold=None, print_each=True):

    dataset = load_iris_data(filepath)
    settings = get_default_settings()

    # TODO 3: Apply fit or threshold overrides
    if threshold is None:
        settings["threshold"] = fit_threshold_from_setosa(
            dataset, settings["threshold"])
    else:
        settings["threshold"] = threshold

    result = initialize_predictions(dataset, settings, print_each=print_each)
    return settings, dataset, result
```

This code show,if user purposely or accidentally did not provide a threshold, the function will automatically learn it from the data using the fit. This makes the function more robust and easier to use, as it can handle cases where the user may not know what threshold to set or simply wants to use a data-driven approach without having to specify every detail manually.

Section 5.6 Detailed Step-by-Step Coding Manual

Follow these steps **in order** inside `session5/session5_iris_template.py`.

Do: Add Imports

Add imports at the very top:

```
import csv
from helper.iris_utils import fit_threshold_from_setosa
```

Do: Update Data Loading to Support Defaults

Change from Session 4 style to Session 5 style:

```
def load_iris_data(filepath=None):
    if filepath is None:
        print("[WARNING] No filepath provided. Using default.")
        filepath = "session5/iris_data.csv"
```

Do: Add Optional Output Control in initialize_predictions

Change the function header to:

```
def initialize_predictions(dataset, settings, print_each=True):
```

Then wrap your print statement inside `if print_each:` so the caller can switch logging on or off.

Do: Upgrade setup_application

Use this Session 5 header:

```
def setup_application(filepath=None, threshold=None, print_each=True, use_fit=False):
```

Apply priority in this order:

1. if `use_fit` is True, learn threshold from `fit_threshold_from_setosa(...)`;
2. else if `threshold` is not None, use the manual threshold override;
3. else keep default threshold from `get_default_settings()`.

Section 5.7 Task: Positional vs Keyword Arguments in main()

Positional Arguments: Python matches arguments to parameters by left-to-right order.

```
# filepath, threshold, print_each
setup_application("session5/iris_data.csv", 1.8, False)
```

Keyword Arguments: You explicitly name the parameter you are setting.

```
setup_application(threshold=2.2, print_each=False)
```

Do: Write main() with Four Runs

In your `main()` function, add a helper `print_summary` to avoid duplicate print statements. Then, write four distinct runs:

1. **Default Run:** `setup_application()`
2. **Positional Run:** `setup_application("session5/iris_data.csv", 1.8, False)`
3. **Keyword Run:** `setup_application(threshold=2.2, print_each=False)`
4. **Fit Run:** `setup_application(print_each=False, use_fit=True)`

Checkpoint

Run the script using `python3 session5/session5_iris_template.py`. You should see 4 separate summaries printed. The default run will print all 50 sample trace lines, while the other runs will not (because `print_each=False`).

Section 5.8 Troubleshooting Mini-Section

Common Mistakes & Errors

- **ModuleNotFoundError:** If Python complains it cannot find `helper`, it means you ran the script from inside the `session5` directory. Navigate to the project root and run it from there.
- **SyntaxError: positional argument follows keyword argument:** In Python, if you mix positional and keyword arguments, all positional arguments **must** come before keyword arguments.
- **TypeError: multiple values for argument:** This happens if you accidentally pass a positional argument that maps to a parameter, and then provide a keyword argument for the exact same parameter.

Section 5.9 Review and Next Steps

Summary:

- **Positional Arguments:** Matched by order. Easy to type, easy to mix up.
- **Keyword Arguments:** Matched by name. Self-documenting and safe.
- **Defaults (`None`):** Allow functions to be called without specifying every parameter, making APIs friendly and backward-compatible.

Review Questions:

1. How do positional and keyword arguments differ?
2. What happens if you pass a keyword argument before a positional argument?
3. Why is it common to use `None` as a default value instead of `0` or `""`?

Micro-Exercise: Change the keyword run in `main()` to use `threshold=1.5` and see how the accuracy changes. Try swapping the order of the keyword arguments (e.g., `print_each=False, threshold=1.5`); does it still work? (Hint: Yes, keyword arguments don't depend on order!)

Stretch Exercise: Add an optional parameter `custom_fit_function=None` to `setup_application`. If provided, use it instead of `fit_threshold_from_setosa`.

Section 5.10 Importing Packages and Functions

In Python, as your project grows, you will want to split your code into multiple files. You can import functions from one file into another using the `import` statement.

For this session, we provide a helper module `session5/helper/iris_utils.py` which contains a function to calculate a threshold from the dataset. You will **not** rewrite this helper; you will **use** it. Import it at the top of your script like this:

```
import csv
from helper.iris_utils import fit_threshold_from_setosa
```

Why this import is important for beginners:

- In Session 4, all logic stayed in one file.
- In Session 5, we start separating logic across files so code is easier to maintain.
- `helper.iris_utils` means: "go to the `helper` folder and use the function inside `iris_utils.py`".

Important: Running from the Project Root

Beginners often think the import path is the same as the file path. In Python, the import path depends on *where you run the script from.* For this tutorial, run your script from the project root: `python3 session5/session5_iris_template.py`. This makes the `helper` module importable because the script looks for `helper` relative to where it was executed.

Your Task (Optional): Refactoring Code for Flexibility Using Keyword Arguments

As an extension of the last session topic about conditional statement (if-else), I have create another scenario where we can apply the if-else statement to determines the antenna color of a robot based on its center_x coordinate and a threshold value. Here is one way on how to solve it:

```
def determine_antenna_color(i, colors, config, my_th):
    if config["center_x"] >= my_th:
        return "purple"
    else:
        return colors[f"robot{i + 1}"]["light_color"]
```

In this part of the code, we initially set the value of `my_th` to 500, and this value was defined within the function as shown below:

```
def initialize_robots(canvas, data):
    colors = data["colors"]
    robot_configurations = data["robot_configurations"]
    condition = True
    my_th = 500
    for i, config in enumerate(robot_configurations):
        create_robot(i, canvas, config, condition, colors, my_th)
```

However, hardcoding values, such as `my_th`, is generally not a best practice in the real-world programming. This is because if the threshold value needs to be changed, you would have to manually update the code each time. A more flexible approach is to pass `my_th` as a parameter. This method allows you to easily adjust the threshold value without needing to alter the code directly.

There are several ways to implement this improvement. For clarity and to focus on understanding keyword arguments, let's make changes starting from the `main()` function. Here's a step-by-step guide on how to refactor the code:

Session 6

Classes and Objects

Section 6.1 Introduction

Goal

Refactor your Session 5 functional pipeline into a class (`IrisRuleClassifier`) that bundles configuration (threshold and labels) with behavior (prediction and evaluation).
What is Object-Oriented Programming (OOP)?

- A **class** is a blueprint that describes what data and behavior a thing should have.
- An **object** is a single instance created from that blueprint.
- An **attribute** is data stored on the object (for example, `self.threshold`).
- A **method** is a function stored on the class that operates on the object (for example, `predict()` and `evaluate()`).

You will work in

Use the following file for this session:

- `session6/session6_iris_template.py` (main student file)

How to run the file: Always run this script from the project root using the terminal command:
`python3 session6/session6_iris_template.py`
Do not edit files outside `session6/`.

In session 4 and session 5, you built an Iris classifier using **functions**. Your pipeline worked, but you may have noticed a design pattern: many functions needed the same configuration values (for example, the threshold and the label names), so you passed a dictionary called `settings` around repeatedly.

Why refactor? That function-based design is fine for one small classifier, but it becomes awkward as soon as you want to run experiments:

- If you want 10 classifiers with 10 thresholds, you now need 10 different `settings` dictionaries.
- Every time you call a function, you must remember to pass the correct `settings`.
- If you add one new setting later, you must update many function signatures and many function calls.

This session (Session 6), you refactor your session 5 design using **Object-Oriented Programming (OOP)**. The core idea is simple:

> **Bundle the data (configuration) and the behavior (functions) together.**

Refactoring promise: the classifier rule itself stays the same. This session reorganizes code, but it should **not change** what "setosa" means or how the threshold rule works.

6.1.1 Transition Map: Session 5 Functions → Session 6 Class

The classifier logic itself (if petal_length < threshold ...) remains exactly the same. What changes is *where* that logic lives and *how* you access it.

Use this table as your translation guide. It shows how session 5 functions and settings translate into session 6 attributes and methods:

Session 5 (Functions + Settings Dict)	**Session 6 (Class + Self State)**	**Explanation of Change**
`get_default_settings()` returns a dict: `{"threshold": 2.0, …}`	`__init__(self, threshold=2.0)`	Instead of passing a dictionary around, we store `self.threshold` inside the object when it is created.
`determine_binary_label(sample, settings)`	`predict(self, sample)`	The function now belongs to the class. It uses `self.threshold` instead of `settings["threshold"]`.
`determine_true_binary_label(sample, settings)`	*(Helper method or part of evaluate)*	Typically logic that calculates "truth" can be a method like `_get_true_label(self, sample)` or inside evaluation.
`initialize_predictions(dataset, settings, print_each)`	`evaluate(self, dataset, print_each)`	The evaluation loop is now a method of the classifier. It iterates over data using its own prediction logic.
`setup_application(filepath, ...)`	`main()` block creating an instance	We replace the procedural setup function with object instantiation: `clf = IrisRuleClassifier(...)`.

Table 6.1: Refactoring Map: From Session 5 Functional Style to Session 6 Object-Oriented Style.

Section 6.2 The Blueprint and Data (`__init__`)

Do: Define the Blueprint and Data (`__init__`)

A class starts with a special constructor method called `__init__`. This method runs **automatically** when you create a new object. Its job is to put the object into a valid starting state.
In session 5, you used `get_default_settings()` to build a dictionary. In session 6, you store that information as **attributes** on `self`. `self` is how a method refers to the current object.

```
class IrisRuleClassifier:
    def __init__(self, threshold=2.0):
        # Store configuration inside the instance (self)
        self.threshold = threshold

        # We enforce these attribute names so later sessions can reuse your class!
        self.positive_label = "setosa"
        self.negative_label = "not_setosa"
```

Task:

1. Define this `__init__` method inside the `IrisRuleClassifier` class.
2. In `main()`, create an instance: `my_clf = IrisRuleClassifier(threshold=2.5)`.
3. Print the stored threshold: `print("Threshold:", my_clf.threshold)`.

Checkpoint

You should be able to create two objects with different thresholds and see that each object stores its own value:

```
clf_a = IrisRuleClassifier(threshold=1.5)
clf_b = IrisRuleClassifier(threshold=2.5)
print(clf_a.threshold, clf_b.threshold)  # should print two different numbers
```

Common Mistake

Forgetting `self`. Inside methods you must write `self.threshold`, not just `threshold`, because `threshold` is not a global variable.

Section 6.3 The Logic (predict)

Do: Implement predict

In session 5, you wrote `classify_sample(sample, settings)`. In session 6, this becomes a **method**. Because `threshold` and the labels already live on `self`, you no longer need a `settings` dictionary argument.
Conceptually, you are changing: `classify_sample(sample, settings)` → `my_clf.predict(sample)`

```
    def predict(self, sample):
        # We assume sample is a dict with 'petal_length'
        if sample["petal_length"] < self.threshold:
            return self.positive_label
        return self.negative_label
```

Task: Add this method to your class (indented inside the class). Identify which inputs are "per sample" and which inputs are "settings". `sample` changes every iteration, so it stays an argument. `threshold` is a setting, so it becomes `self.threshold`.

Checkpoint

Call `my_clf.predict(...)` on a single dictionary and make sure you get either `"setosa"` or `"not_setosa"`.

Common Mistake

Forgetting to include `self` as the first parameter of a method. If you define `def predict(sample):`, Python will raise a TypeError when you call it!

Section 6.4 Processing the Dataset (Evaluation)

Do: Implement evaluate

In session 5, you had a function `initialize_predictions` that looped over the data. We will now create a method `evaluate(self, dataset)` that does the entire prediction loop and calculates accuracy.
Why make this a method? Because evaluation depends on `predict()`, and `predict()` depends on the object's configuration. By keeping them together, you avoid passing settings around.
Task: Implement the `evaluate(self, dataset)` method.

- The evaluation loop should call `self.predict(sample)`.
- Determine truth (if species is "setosa", truth is `self.positive_label`, else `self.negative_label`).
- The method should keep counting logic local (`correct`, `total`).
- **CRITICAL:** Ensure `total += 1` runs for every sample, even if the prediction is wrong.
- Return accuracy (`correct / total` if `total > 0` else `0.0`).

Checkpoint

- Accuracy is a number between 0.0 and 1.0.
- Test your method on a dataset with at least one misclassified sample and confirm that `total` equals the number of samples.

Common Mistake

Forgetting to increment `total` for every sample, even wrong ones. If you only increment total when correct, your accuracy will always be 100% (which is wrong!).

Section 6.5 Putting it all together (Main Script)

Do: Use the Class in `main()`

Now you can use your class to test different classifiers easily. The workflow becomes:

1. Use the provided `dataset` (list of dictionaries).
2. Create one or more classifier objects (each with its own threshold).
3. Ask each object to evaluate itself on the dataset.

```
# Create Objects with different configurations
clf_strict = IrisRuleClassifier(threshold=1.5)   # Strict threshold
clf_loose  = IrisRuleClassifier(threshold=2.5)   # Loose threshold

# Evaluate each one
print("Strict Classifier Accuracy:", clf_strict.evaluate(dataset))
print("Loose Classifier Accuracy: ", clf_loose.evaluate(dataset))
```

Notice that the dataset is passed in, but the threshold is not. The threshold lives inside each object!

Section 6.6 Review and Next Steps

Summary:

- **Classes** group data (attributes) and behavior (methods) together.
- **Objects** remember their own configuration, allowing you to create many variations easily.
- Object-oriented design is the foundation of reusable software libraries. Machine-learning libraries like `scikit-learn` use objects to encapsulate models!

Review Questions:

1. What does `self` refer to inside a method?
2. Why did we move `threshold` from a dictionary into `self.threshold`?
3. Why is inheritance useful? (Think ahead: what if we wanted a new classifier that guessed randomly, but shared the same evaluation logic?)

Micro-Exercise: Create two classifier objects with different thresholds (e.g., 1.4 and 5.0) and compare their accuracies. Can you find a threshold that performs perfectly on the provided dataset?

Stretch Exercise: Write an additional method on the class called `predict_all(self, dataset)` that returns a list of predictions for the entire dataset, rather than just returning accuracy.

Session 7

Inheritance and Polymorphism

Section 7.1 Introduction

In session 6, we bundled our code into a single class: `IrisRuleClassifier`. It had two main jobs:

1. **Decide:** Does the sample have `petal_length < threshold`? (This is specific to this rule).
2. **Evaluate:** Loop through the dataset and calculate accuracy. (This is general).

You will work in

Use the following file for this session:

- `session7/session7_iris_template.py` (main student file)

How to run the file: Always run this script from the project root using the terminal command: `python3 session7/session7_iris_template.py`

So far, having one class is already an improvement over passing a `settings` dictionary everywhere. But a new design problem appears as soon as you want **more than one type** of classifier.

What if we want to create a new classifier, such as a `NearestCentroidClassifier` (a different prediction strategy) or even a simple `RandomGuessClassifier`? With the session 6 design, every new class would need:

- its own `predict(...)` logic (different for each classifier type),
- and a copy of the same `evaluate(...)` loop (identical across classifier types).

Why is that a problem?

- Copy-paste creates duplication. Duplication increases bugs.
- If you fix a mistake in `evaluate()`, you must remember to fix it in every class.
- Adding a new classifier becomes "rewrite everything again" instead of "write only what is different".

This session (Session 7), we introduce **Inheritance**. We will extract the shared logic (evaluation) into a **Parent Class** (Base), and keep only the specific logic (prediction rules) in the **Child Classes**. **Why introduce inheritance now?** Because you now have enough experience to see the repeating pattern: many classifiers differ in `predict()`, but they can share `evaluate()`. Inheritance is one clean way to reuse that shared behavior.

Real-World Hook

Inheritance is like defining a base concept of a "Vehicle" that has wheels and moves, and then defining child concepts like "Car" and "Bike" that inherit the basic behaviors but add their own specific rules (like having an engine vs. pedals).

7.1.1 Key Terms

Before we write code, remember these concepts:

- **Sample:** A dictionary representing one flower, e.g., `{"petal_length": 1.4, "species": "setosa"}`. Notice that keys like `"petal_length"` and `"species"` are strings.
- **Dataset:** A list of sample dictionaries.

7.1.2 Transition Map: Session 6 Class → Session 7 Inheritance

Session 6 (Standalone Class)	Session 7 (Inheritance Hierarchy)	Explanation of Change
`IrisRuleClassifier.evaluate(self, ...)`	`ClassifierBase.evaluate(self, ...)`	Every classifier needs evaluation. We move this shared logic up to the parent **Base Class** so all children get it for free.
`IrisRuleClassifier.predict(self, ...)`	`ChildClass.predict(self, ...)`	Prediction logic is specific to each algorithm. The parent defines the signature, but the **Child Class** provides the specific implementation.
`__init__`	`super().__init__()`	Child classes call the parent's constructor to ensure shared setup (like logging names) is done correctly.
One single class file	A hierarchy of classes	We can now easily add new classifiers (`NearestCentroid`, `RandomGuess`) without rewriting the evaluation loop.

Table 7.1: Refactoring Map: From Single Class (S6) to Parent/Child Inheritance (S7).

Section 7.2 The Parent Class

Do: Create the Parent Class (`ClassifierBase`)

This class will hold the logic that is *shared* by all classifiers. For us, that is the `evaluate` method. It doesn't know *how* to predict yet, but it knows how to *score* predictions.

```
class ClassifierBase:
    def __init__(self):
        pass

    def predict(self, sample):
        # The parent doesn't know how to predict!
        # Child classes MUST replace this method with their own logic.
        raise NotImplementedError("Child class must implement predict()")

    def evaluate(self, dataset):
        # This logic is shared! It calls self.predict(), provided by the child.
        correct = 0
        total = 0

        for sample in dataset:
            prediction = self.predict(sample)

            # Simple binary truth check
            truth = "setosa" if sample["species"] == "setosa" else "not_setosa"

            if prediction == truth:
                correct += 1
            total += 1  # BUG FIX: Ensure total increments unconditionally!

        return correct / total if total > 0 else 0.0
```

Task: Copy this parent class into your file.

Before you code (reasoning): decide what should be shared across classifiers. Evaluation is shared (loop, counting, accuracy). Prediction is not shared (rules differ), so the parent class should *not* try to implement it. What would happen if you forgot to override `predict()` in a child class? You would get a `NotImplementedError`!

Checkpoint

Create an instance of `ClassifierBase` and call `evaluate(dataset)` on a small dataset. You should get a `NotImplementedError` because `predict()` is not implemented.

Section 7.3 Inheritance and Child Classes

Do: The First Child (`RuleClassifier`)

Now we recreate our session 6 classifier, but this time it **inherits** from `ClassifierBase`. It only needs to provide the `__init__` and `predict` logic. It gets `evaluate` for free!

```
class RuleClassifier(ClassifierBase):  # <--- Inherits from ClassifierBase
    def __init__(self, threshold=2.0):
        super().__init__()  # Initialize the parent
        self.threshold = threshold

    def predict(self, sample):
        # Specific logic for THIS classifier
        if sample["petal_length"] < self.threshold:
            return "setosa"
        return "not_setosa"
```

Task:

1. Define this class below `ClassifierBase`.
2. Create an instance: `rule_clf = RuleClassifier(2.0)`
3. Call `print(rule_clf.evaluate(dataset))`

Checkpoint

Notice that you can call `rule_clf.evaluate(dataset)` even though you did not write `evaluate` inside `RuleClassifier`. That is the practical benefit of inheritance.

Do: The Second Child (`NearestCentroidClassifier`)

We can add a completely different classifier without rewriting the evaluation loop. This classifier classifies a sample by comparing its `petal_length` to two target values and picking the closer one.

```
class NearestCentroidClassifier(ClassifierBase):
    def __init__(self, setosa_target=1.4, non_setosa_target=4.5):
        super().__init__()
        self.setosa_target = setosa_target     # Target petal_length for setosa
        self.non_setosa_target = non_setosa_target # Target for others

    def predict(self, sample):
        # Logic: Which target value is the sample's petal_length closer to?
        val = sample["petal_length"]
        dist_to_setosa = abs(val - self.setosa_target)
        dist_to_non = abs(val - self.non_setosa_target)

        if dist_to_setosa < dist_to_non:
            return "setosa"
        return "not_setosa"
```

Task:

1. Define this class.
2. Create an instance: `centroid_clf = NearestCentroidClassifier()`
3. Call `print(centroid_clf.evaluate(dataset))`

Checkpoint

Evaluate this classifier on the same dataset without writing any new evaluation code!

Section 7.4 Polymorphism

Do: Putting it all together

We can loop through a list of *different types* of classifiers and treat them all the same because they share the same Parent Interface. This is called **polymorphism**.

```
# Create a list of different classifiers
classifiers = [
    RuleClassifier(threshold=2.0),
    NearestCentroidClassifier(),
]

# Evaluate them all in one loop
print("=== Model Comparison ===")
for clf in classifiers:
    acc = clf.evaluate(dataset)
    # Uses the class name to identify which one ran
    print(f"{clf.__class__.__name__}: Accuracy = {acc}")
```

What is happening conceptually?

- Each object has a different `predict()` implementation.
- They all share the same `evaluate()` implementation from the parent class.
- The loop does not need to know the exact class type. It only needs to know that the object supports `evaluate()`.

Section 7.5 Troubleshooting

Common Mistakes & Troubleshooting

- **Forgetting to inherit:** If you accidentally write `class RuleClassifier:` (missing `(ClassifierBase)`), you will not get `evaluate()` for free. Python will raise an `AttributeError`.
- **Forgetting `super().__init__()`:** Always initialize the parent class in the child's constructor!
- **Label mismatches:** If your `predict()` returns "SETOSA" instead of "setosa", it won't match the `evaluate()` loop's logic, and you'll get 0% accuracy.

Section 7.6 Review and Next Steps

Summary:

- **Inheritance** allows child classes to reuse code from a parent class, eliminating duplication.
- **Polymorphism** allows you to treat different objects uniformly if they share a common interface.
- Shared logic lives in one place (`ClassifierBase.evaluate`), while new classifier types only implement what is unique (`predict()`).

Reflection Questions:

1. How does polymorphism allow you to add new classifiers easily?
2. What happens if the parent class does not raise `NotImplementedError` and a child forgets to define `predict()`?

Optional Exercise (RandomGuessClassifier): Implement a third classifier called `RandomGuessClassifier` that inherits from `ClassifierBase`. It should ignore the sample entirely and randomly return "setosa" or "not_setosa". Add it to your `classifiers` list in `main()` and observe its accuracy!

7.6.1 Submission Requirement

Add your student label at the end:

```
print("=== Student Label ===")
print("ID: ...")
```

Run the script passing the dataset to both classifiers.

Session 8

NumPy (Iris — session 8)

Section 8.1 Introduction

In session 6 and session 7, we focused on **classes** and **inheritance** to build reusable classifiers for the Iris classification task. This session, we introduce **NumPy**, the most important Python library for fast numerical computing. NumPy is designed to work efficiently with large arrays of numbers, which is exactly what we need for machine learning.

Recall from session 7 that our classifier interface was `fit()`, `predict_one()`, `predict_all()`, and `evaluate()`. In this session, we keep that interface but implement NumPy-based data operations underneath it.

You will work in

Use the following files for this session:

- `session8/session8_iris_template.py` (main student file)
- `session8/iris_session8_utils.py` (data loading and utilities)
- `session8/iris_session8_models.py` (NumPy-based classifier model)

How to run the file: Run from the project root:
`python3 session8/session8_iris_template.py`

8.1.1 What You Will Learn

By the end of this tutorial, you will be able to:

- create NumPy arrays and understand their advantages over Python lists for numerical tasks,
- use indexing and slicing on arrays,
- compute means with `np.mean(..., axis=…)`,
- use boolean masks to select rows in a dataset,
- compute Iris class centroids and use them for classification intuition,
- import custom functions/classes from one Python file to another.

8.1.2 Tools and Setup

You need:

- Python 3,
- NumPy installed (`pip install numpy`),
- an `iris.csv` file.

Assumption about `iris.csv`: it contains 5 columns:

```
sepal_length,sepal_width,petal_length,petal_width,species
```

If your file has a header row, we will handle it by skipping the first row.

Section 8.2 Understanding X and y

In machine learning, data is usually separated into features and labels.

- **X:** A 2D NumPy array of shape `(n_samples, 4)` containing the numeric features (sepal length, sepal width, petal length, petal width).
- **y:** A 1D NumPy array of shape `(n_samples,)` containing the string species labels.

8.2.1 Activity 1: Load Iris into NumPy arrays (X and y)

Create a new file called `session8/iris_session8_utils.py`. Start with:

```
import numpy as np

def load_iris_csv(filepath="iris.csv", skip_header=1):
    """
    Loads iris.csv into:
      X: (n_samples, 4) float array
      y: (n_samples,) string array
    """
    # Use genfromtxt for mixed types (numbers + strings)
    data = np.genfromtxt(
        filepath,
        delimiter=",",
        dtype=None,          # allow mixed dtype
        encoding="utf-8",
        skip_header=skip_header
    )

    # data will be a structured array with 5 columns
    X = np.column_stack([data[i] for i in range(4)]).astype(float)
    y = np.array(data[4], dtype=str)

    return X, y
```

Now in `session8/session8_iris_template.py`, write:

```
import numpy as np
from session8.iris_session8_utils import load_iris_csv

X, y = load_iris_csv("iris.csv")
print("X shape:", X.shape)
print("X dtype:", X.dtype)
print("y shape:", y.shape)
print("y dtype:", y.dtype)
```

Checkpoint

Run `session8_iris_template.py`. Confirm you see `X shape: (150, 4)` and `y shape: (150,)`. Check that `X dtype` is `float64` and `y` is `<U10` (string).

8.2.2 Activity 2: Quick NumPy warm-up (indexing and slicing)

Add the following to `session8_iris_template.py` below your prints:

```
# First sample (row 0)
print("First row of X:", X[0])
print("First label y[0]:", y[0])

# First 5 rows
print("First 5 rows of X:\n", X[:5])

# Only petal features (columns 2 and 3)
petal = X[:, 2:4]
print("Petal matrix shape:", petal.shape)

# Sepal length column only (column 0)
sepal_length = X[:, 0]
print("Sepal length shape:", sepal_length.shape)
```

Do: Your Task

1. What will `X[-1]` return? Add a print statement to output the last row using negative indexing.
2. Print the first 10 labels from `y`.

Checkpoint

Verify your last row matches the bottom of your CSV and your first 10 labels are all `setosa`.

8.2.3 Activity 3: Compute centroids (mean feature vector) per species

A **boolean mask** is an array of True/False values that you can use to select specific rows from another array. For example, `y == "setosa"` returns an array of booleans. We can then do `X[y == "setosa"]` to grab only the rows in `X` where the species is setosa.

A **centroid** is the mean of the features for a class. For each species, we want:

$$\mu_{\text{class}} = \texttt{mean}(X_{\text{class}}, \text{axis} = 0)$$

This gives a 4D vector: `[mean_sepal_length, mean_sepal_width, mean_petal_length, mean_petal_width]`.

Step 1: Find unique species

```
classes = np.unique(y)
print("Classes:", classes)
```

Step 2: Compute centroid with a boolean mask

```
centroids = {}

for c in classes:
    mask = (y == c)          # boolean array, True where class matches
    X_c = X[mask]            # select rows belonging to class c
    mu = np.mean(X_c, axis=0)
    centroids[c] = mu

print("Centroids:")
for c in classes:
    print(c, "->", np.round(centroids[c], 2))
```

Checkpoint

Verify that the `centroids` dictionary contains three entries (one per species) and each entry is an array of 4 rounded numbers.

8.2.4 Activity 4: Why `axis=0` matters

In NumPy, `axis=0` means: "collapse rows (samples) and compute the mean per column (feature)". `axis=1` means "collapse columns and compute the mean per row (sample)".

```
example = X[:3]  # first 3 samples
print("example shape:", example.shape)
print("mean axis=0 (per feature):", np.mean(example, axis=0))
print("mean axis=1 (per sample):", np.mean(example, axis=1))
```

Do: Your Task

1. In one sentence (as a comment in your code), explain the difference between `axis=0` and `axis=1`.

Section 8.3 Troubleshooting Imports and CSV Loading

Common Mistakes & Troubleshooting

- **ModuleNotFoundError:** If Python cannot find `session8.iris_session8_utils`, make sure you are running the script from the project root directory, not from inside the `session8` folder.
- **FileNotFoundError or ValueError (CSV):** If `load_iris_csv` fails, check that `iris.csv` exists. If you get a ValueError about parsing strings, ensure `skip_header=1` matches your CSV file.

8.3.1 Activity 5: Code organization across files (imports)

As projects grow, we avoid putting everything in one file. Practice splitting code into three files: `iris_session8_utils.py`, `iris_session8_models.py`, and `session8_iris_template.py`.

Recall from session 7 that the classifier interface should stay stable. In `iris_session8_models.py`, we define a `NumpyCentroidClassifier` class that keeps the same method names: `fit()`, `predict_one()`, `predict_all()`, and `evaluate()`.

Do: Your Task

1. Run `session8/session8_iris_template.py` and confirm the imports work and the centroid classifier correctly evaluates the dataset.
2. Intentionally rename `session8/iris_session8_utils.py` to something else and observe the import error.
3. Rename it back correctly.

This file/module structure is important because Session 9 will import these exact same utilities and model interface for plotting-based analysis.

8.3.2 Optional Activity: Use an LLM to explain NumPy-heavy code

Professional developers often paste unfamiliar code into an LLM and ask for a guided explanation. This is a **learning tool**, not a replacement for understanding.

Prompt 1: Explain NumPy usage

Copy your centroid computation code and ask:

> Explain every place NumPy is used in this script. For each NumPy function, tell me: (1) what it does, (2) what shape the input is, (3) what shape the output is.

Prompt 2: Explain boolean masks + axis mean

Copy the loop and ask:

> Explain line-by-line how boolean masking works in NumPy here, and why axis=0 produces a centroid vector.

Do: Reflection

Write 3–5 lines (as comments in your Python file):

- One thing the LLM explained clearly.
- One thing you still find confusing.

Section 8.4 Summary and Reflection

Summary:

- **NumPy Arrays:** Provided a more efficient, faster way to store numerical data than Python lists. We explored shapes and dtypes.
- **Boolean Masking:** Allowed us to filter data without writing explicit `for` loops.
- **Axes:** `axis=0` computes operations across rows (giving per-column results), while `axis=1` computes across columns.
- **Modularization:** Code was successfully split across utilities, models, and execution scripts.

Reflection Questions:

1. Why does NumPy make numerical operations faster than Python lists?
2. What does `axis=0` mean in the context of computing centroids for the Iris dataset?

8.4.1 Submission Requirement (Graded)

Submit:

1. `session8/session8_iris_template.py`
2. `session8/iris_session8_utils.py`
3. `session8/iris_session8_models.py`
4. A screenshot/PDF of your terminal output showing `X shape` and `X dtype`, and printed centroids.

Session 9

Data Visualization and Experimentation (Iris)

Section 9.1 Introduction

In this session, we shift fully into an **Iris-only** workflow. Recall from Session 8 that we already produced NumPy arrays (`X`, `y`) and class centroids. Now, building on Session 8's outputs, we visualize classifier behavior and experiment results.

You will work in

For this session, you will create or edit three files in the `session9/` folder:

- `session9/iris_eval.py` (helper functions for train/test split and accuracy)
- `session9/session9_iris_template.py` (main experiment runner)
- `session9/session9_plot_accuracy.py` (script for plotting)

How to run: From the project root:
`python3 session9/session9_iris_template.py`
`python3 session9/session9_plot_accuracy.py`

You will work on two key tasks:

1. **Systematic parameter experiments:** run repeated experiments by changing a classifier setting (e.g., distance metric, threshold, or test size), and record the accuracy.
2. **Visualization:** plot how performance changes across settings, using box plots and line plots.

This tutorial is important because good data practice means:

- running **multiple trials** to reduce randomness and draw reliable conclusions,
- saving results in a structured form (**list of dictionaries** → **DataFrame**),
- visualizing distributions (box plot) and trends (line plot).

9.1.1 Prerequisites

You need:

- `numpy`, `pandas`, `matplotlib`, `openpyxl`
- an `iris.csv` file

Install if needed:

```
pip install numpy pandas matplotlib openpyxl
```

9.1.2 Calculating Accuracy for Iris Experiments

In session 9, we combine prior ideas into an **experiment pipeline**:

- load Iris into NumPy arrays,
- split the data into training and test sets,
- run a classifier,
- compute accuracy,
- repeat under different settings,
- store and visualize results.

Important Parameters:

- `test_size`: The proportion of data used for evaluating the model. A larger test size leaves less data for training.
- `seed`: Initializes the random number generator. Using a seed ensures our random data splits are reproducible.
- `metric`: The distance metric used by the centroid classifier (`"l2"` is Euclidean distance, `"l1"` is Manhattan distance).

Your Task: Reuse session 8 Modules

Session 9 should import from Session 8 (e.g., `session8.iris_session8_utils` and `session8.iris_session8_models`) to preserve continuity.

9.1.3 Activity 1: Implement Train/Test Split and Accuracy

Do: Create `session9/iris_eval.py`

We will write a custom split function using only NumPy:

```
# session9/iris_eval.py
import numpy as np

def train_test_split(X, y, test_size=0.2, seed=1):
    """
    NumPy-only train/test split.
    """
    n = X.shape[0]
    rng = np.random.default_rng(seed)
    idx = np.arange(n)
    rng.shuffle(idx)

    test_n = int(n * test_size)
    test_idx = idx[:test_n]
    train_idx = idx[test_n:]

    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]

def accuracy_score(y_true, y_pred):
    return float(np.mean(y_true == y_pred))
```

9.1.4 Activity 2: Refactoring for Flexibility

A common beginner mistake is hardcoding values:

```
# BAD PRACTICE (example)
test_size = 0.2
seed = 1
metric = "l2"
```

This makes it hard to run systematic experiments. Instead, we'll create a function that takes these as arguments.

Do: Update `session9/session9_iris_template.py`

Create the main experiment runner:

```
# session9/session9_iris_template.py
import numpy as np
from session8.iris_session8_utils import load_iris_csv
from session9.iris_eval import train_test_split, accuracy_score
from session8.iris_session8_models import NumpyCentroidClassifier

def run_experiment(test_size=0.2, seed=1, metric="l2"):
    X, y = load_iris_csv("iris.csv", skip_header=1)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size,
    ↪  seed=seed)

    model = NumpyCentroidClassifier(metric=metric)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    return acc

if __name__ == "__main__":
    acc = run_experiment(test_size=0.2, seed=1, metric="l2")
    print(f"One run | test_size=0.2 | seed=1 | metric=l2 | accuracy={acc:.3f}")
```

Checkpoint

Run the file and ensure you get an accuracy output. Then, change the metric to `"l1"` and run it again to see if the accuracy changes.

9.1.5 Running Multiple Experiments Dynamically

Now we want to run multiple experiments with different parameter settings.

Do: Implement Looped Output

Add this function inside `session9/session9_iris_template.py`:

```
def run_single_setting_sweep(metrics, seeds, test_size=0.2):
    print(f"--- Sweep: test_size={test_size} ---")
    for metric in metrics:
        for seed in seeds:
            acc = run_experiment(test_size=test_size, seed=seed, metric=metric)
            print(f"metric={metric} | seed={seed} | acc={acc:.3f}")
```

Then call it in the main block with `metrics=["l1", "l2"]` and `seeds=[1, 2, 3, 4, 5]`.

Checkpoint

Run a small sweep. You should see 10 lines of output representing the combination of 2 metrics and 5 seeds.

9.1.6 Activity 3: Good Scientific Practice - Multiple Trials

We run **multiple trials** for each configuration because randomness in the train/test split can cause accuracy to fluctuate. Averaging over many trials yields more reliable estimates.

Do: Implement `run_multiple_trials`

```
def run_multiple_trials(metrics, test_sizes, num_trials):
    """
    Returns a list of dicts:
    {'metric':..., 'test_size':..., 'trial':..., 'seed':..., 'accuracy':...}
    """
    results_list = []

    for metric in metrics:
        for test_size in test_sizes:
            print(f"\n[Config] metric={metric} | test_size={test_size}")
            for trial in range(1, num_trials + 1):
                seed = trial  # simple choice: seed=1..num_trials
                acc = run_experiment(test_size=test_size, seed=seed, metric=metric)

                print(f"  Trial {trial}/{num_trials} | seed={seed} | acc={acc:.3f}")

                run_data = {
                    "metric": metric,
                    "test_size": test_size,
                    "trial": trial,
                    "seed": seed,
                    "accuracy": acc
                }
                results_list.append(run_data)

    return results_list
```

9.1.7 Activity 4: Analyzing Results with Pandas

DataFrames allow us to group, summarize, and easily save tabular data.

Do: Use Pandas to Process and Save Results

At the top of `session9_iris_template.py`, add:

```
import pandas as pd
```

In `main()`, after generating `all_results_data`:

```
results_df = pd.DataFrame(all_results_data)
print(results_df.head())

output_filename = "iris_experiment_results.xlsx"
results_df.to_excel(output_filename, index=False)
print(f"Saved: {output_filename}")
```

Checkpoint

Check your project directory for the newly created `iris_experiment_results.xlsx` file.

Section 9.2 Troubleshooting Imports and Excel

Common Mistakes & Troubleshooting

- **ModuleNotFoundError:** Ensure you are running from the project root.
- **PermissionError (Excel):** If you get a ”file in use” error when running your script, make sure you close the Excel file before trying to overwrite it!

9.2.1 Activity 5: Analyzing and Visualizing Results

Box plots show the distribution of accuracies, letting you see the median and variance. **Line plots** show the trend, such as how accuracy drops as test size increases.

Do: Visualization with Matplotlib

Create `session9/session9_plot_accuracy.py`:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_excel("iris_experiment_results.xlsx")

# Box plot: accuracy distribution by metric
df.boxplot(column="accuracy", by="metric")
plt.title("Accuracy Distribution by Distance Metric")
plt.suptitle("")
plt.xlabel("Metric")
plt.ylabel("Accuracy")
plt.grid(True)
plt.savefig("session9_boxplot.png")
plt.show()

# Line plot: mean accuracy vs test_size
avg = df.groupby(["metric", "test_size"])["accuracy"].mean().reset_index()

plt.figure()
for metric in sorted(avg["metric"].unique()):
    sub = avg[avg["metric"] == metric]
    plt.plot(sub["test_size"], sub["accuracy"], marker="o", label=metric)

plt.title("Mean Accuracy vs Test Size")
plt.xlabel("Test Size")
plt.ylabel("Mean Accuracy")
plt.grid(True)
plt.legend()
plt.savefig("session9_lineplot.png")
plt.show()
```

Section 9.3 Summary and Reflection

Summary:

- We defined parameters like `test_size` and `seed` to systematically test models.
- **Pandas** helped us collect dictionaries into DataFrames for easy aggregation and export.
- **Matplotlib** enabled us to visualize distributions and trends across metrics.

Reflection Questions:

1. How does changing the `test_size` impact the overall accuracy, according to your line plot?
2. Why do we randomize seeds instead of using the same seed for every trial?

9.3.1 Submission

Submit:

- `session9/session9_iris_template.py`
- `session9/session9_plot_accuracy.py`
- `session9/iris_eval.py`
- `iris_experiment_results.xlsx`
- screenshots of plots (box plot + line plot)